SYSTEM DESIGN INTERVIEW PLAYBOOK

A structured method for designing systems under interview pressure

AH Architecture Lab

Version 1.0

Contents

System Design Interview Playbook

A structured method for designing systems under interview pressure

AH Architecture Lab

Version 1.0 — *Writing*

Copyright

Status: Writing
Edition: Version 1.0 (manuscript)

Publisher: AH Architecture Lab

Dedication

For the engineers who have to think clearly with a marker in one hand and a clock in the other.

Foreword

Placeholder for Version 1.0. Invite a practitioner who has hired for senior and architect roles to write a one-page note on why a repeatable design method matters more than a catalog of products.

Preface

I wrote this playbook as an enterprise architect who has sat on both sides of the table: shipping platforms, and listening to candidates design them in forty-five minutes.

The book has one product: a **repeatable method**. Building blocks and worked problems exist to practice that method, not to be memorized as the “correct” box diagram.

The manuscript is Markdown in Git so chapters, Mermaid, and Draw.io files can be revised like code. Version 1.0 opens with ten Part I foundation chapters — from requirements and capacity through CAP, consistency, and trade-offs — then building blocks and a small set of interview problems. Later editions can add problems without changing Part I.

Every chapter uses the same spine — objectives, concepts, a diagram, a real example, an enterprise note, the interviewer’s ear, an AI note, mistakes, practices, takeaways, questions — so you always know where you are.

AH Architecture Lab

Acknowledgements

To be written at freeze. Thank reviewers who red-lined diagrams, interviewers who sharpened the method, and readers who reported ambiguities.

Part I — Foundations

Welcome to the World of System Design

Learning Objectives

- Explain what a system design interview is actually scoring.
- Distinguish a repeatable method from a memorized product catalog.
- Recognize the chapter shape used for the rest of this book.
- Name the five layers that belong on a first-pass architecture sketch.

Quote

> *A design interview is a short, public architecture review. The marker is not there to decorate the board. It is there to make your trade-offs visible.*

Introduction

If you have built production systems, you already know more than the interview requires. The failure mode is not ignorance. It is **unstructured competence**: jumping to Kafka, then to Kubernetes, then to a multi-region story before anyone agreed what "the system" is.

This book treats the interview as a design review with a clock. **Part I** is ten foundation chapters: how to think before you draw, how to estimate, then scale, availability, reliability, performance, CAP, consistency, and the trade-off sentence that ties them together. Later parts add building blocks and worked problems. You will practice a loop: bound the problem, make a few honest numbers, draw one picture, go deep where it hurts, and close.

Three trade-off pairs show up in almost every strong board. Remember them as a preview, not as slogans: **performance versus scalability** (Chapter 4 and 7), **latency versus throughput** (Chapter 7), **availability versus consistency** (Chapters 5, 8, and 9). Chapter 10 is how you say which pair you spent.

Core Concepts

Three ideas sit under every chapter.

Method over inventory. Interviewers can buy cloud products. They cannot buy your judgment. Judgment shows up as a sequence: what you asked, what you assumed, what you drew, and what you would revisit if a constraint moved.

One picture, two paths. A useful board has a read path and a write path. If you cannot trace a user click from the edge to durable storage and back, you do not yet have a design. You have a collage.

Trade-offs are the payload. Every box you add costs money, latency, or operational load. Saying “we will add a cache” is unfinished. Saying “we cache the mapping because the working set is small and the read/write ratio is extreme; we accept stale redirects for a few seconds” is a design.

Architecture Diagram

A first-pass sketch almost always has the same five layers. Rename the boxes. Do not skip a layer without a sentence.

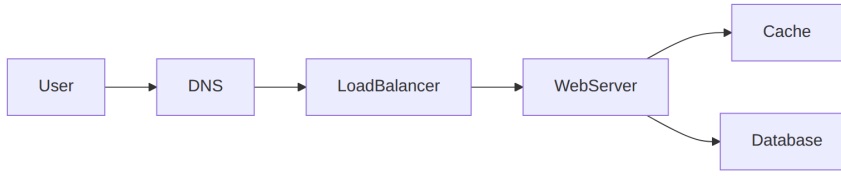


Figure 1.1 — Default request path. DNS and the load balancer are the edge; the web server is application logic; cache and database are data. Asynchronous work is missing on purpose: add it when the user should not wait.

Real-world Example

You are asked to “design a document store for a company of 40,000 people.” A weak start lists S3, Elasticsearch, and a service mesh. A strong start asks: are we storing contracts with retention rules, or wiki pages with live collaboration? One of those is an object plus metadata and a search projection. The other is a consistency problem with presence and conflict. Same English word, different diagrams. The welcome chapter’s job is to make you pause long enough to hear the difference.

Enterprise Insight

In an enterprise, the interview is a proxy for architecture reviews you will attend weekly: security wants a control, finance wants a cost line, a platform team wants you on the blessed load balancer, and a product owner wants the feature Friday. Your value is not drawing more boxes. It is naming which constraint is allowed to win today, and which decision is reversible next quarter.

Interviewer’s Mind

The interviewer is asking: *Can I put this person in a room with a messy problem and a senior stakeholder without being embarrassed?* They listen for structure, not for the newest product name. If you skip requirements, they will drag you back. If you never draw, they will not trust the words. If you never say what you are giving up, they will assume you have not noticed.

AI Perspective

Generative tools can dump a plausible architecture in seconds. That is a hazard in the room. Interviewers are scoring *your* sequence of questions and trade-offs. Use models after the interview to critique your own diagram. In production, AI features are just more services: they add GPU capacity, prompt/

versioning, data-retention questions, and a latency budget that does not belong on the user's click path unless the product is the model.

Common Mistakes

- Starting with technology names before a problem sentence.
- Drawing ten products and no request path.
- Treating the interview as a quiz on one famous book or course.
- Ignoring the clock, then rushing the only part the interviewer cared about.

Best Practices

- Restate the problem and the non-goals out loud.
- Keep a parking lot for "if we have time."
- Draw early; talk while you draw.
- When you add a component, say the number or NFR that justified it.

Summary

This book is a playbook, not an encyclopedia. You will see the same chapter shape until it is muscle memory. The default diagram is five boxes and two data stores. Everything else is a variation you must justify.

Key Takeaways

- Interviews score structured judgment under time.
- Method first, inventory second.
- Always show a read path and a write path.
- Name the trade-off when you add a box.

Interview Questions

- What is the interviewer actually hiring for in a system design round?
- What belongs on a first-pass diagram before any vendor name?
- How do you recover if you realize ten minutes in that you designed the wrong product?

Further Reading

- Part I continues: requirements (Chapter 2) through architectural trade-offs (Chapter 10).
- This book's STANDARDS.md for chapter and diagram rules.
- Appendix A for the board cheat sheet.
- Your last production design review notes: rewrite them as a 45-minute board session.

Understanding requirements

Learning Objectives

- Separate functional requirements from non-functional ones that change the diagram.
- Capture constraints, assumptions, and scope before any boxes.
- Ask clarifying questions that unlock architecture, not trivia.
- Prioritize must-have versus nice-to-have so a 45-minute design stays honest.
- Walk a real prompt (“design a WhatsApp-like messenger”) the way an architect would, before drawing.

Quote

> *Architects do not start with boxes. They start with verbs, limits, and the questions that make a slogan into a system.*

Introduction

Every system design interview starts here, whether the interviewer says so or not. “Design WhatsApp.” “Design a news feed.” “Design a payments API.” Those are slogans. If you reach for a load balancer in the first sixty seconds, you are guessing.

This chapter is how architects think **before** they design: functional behavior, non-functional pressure, constraints you cannot negotiate, assumptions you must say out loud, clarifying questions, a scoped MVP, and an explicit parking lot. Capacity numbers come in Chapter 3. The picture comes after that. Requirements are the filter that decides what the picture is allowed to contain.

Core Concepts

Functional requirements

Functional requirements are user-visible verbs. Send a message. Receive it while offline. Create a group. They belong in language a product owner would recognize. They are not “we will use WebSockets.”

Write them as a short list. If you cannot demo the verb, it is not functional yet.

Non-functional requirements

Non-functional requirements (NFRs) are how the verbs must feel: latency, availability, consistency, durability, security, cost, compliance, operability. You cannot maximize all of them. Circle the two that would **redraw the diagram** if they moved.

“End-to-end encryption” is an NFR that can split your design (where keys live, what the server is allowed to see, how search works). “Pretty bubbles” is not.

Constraints

Constraints are facts you are not allowed to wish away: existing identity provider, data residency, a mobile-only client, a peak event next month, a team of four. In interviews they often arrive as “you cannot use a managed queue” or “assume we already have object storage.” Write them on the board. Fighting a stated constraint to look clever is a fail.

Assumptions

Assumptions are requirements you invented because nobody knew. They are valid only if you **speak them**. “I will assume 50 million daily active users and one-to-one chat first; tell me if that is wrong.” Silent assumptions become silent failures when the interviewer meant 50 thousand internal users and groups of 5,000.

Clarifying questions

Good questions change a box, an SLO, or the data model. Bad questions collect trivia. Prefer:

- Who is the user, and what is the one verb they must complete?
- What are the inputs and outputs of that verb?
- How much data do we store, and for how long?
- Rough requests per second, and the **read/write ratio**?
- One-to-one only, or groups? How large?
- Online-only, or store-and-forward?
- Media? Voice or video in this round?
- Read receipts, typing indicators — MVP or later?
- Encryption: transport only, or end-to-end?
- Daily active users, peak vs average, read/write shape?
- How long do we retain messages? Who may search them?
- Which region(s)? Any regulation on where bytes sit?

Inputs, stored volume, RPS, and read/write ratio are not trivia. They are the first four cells of the Chapter 3 table. Ask them even when the prompt is a brand name.

Stop when the next question would not change the first diagram.

Scope definition

Scope is an explicit in-list and out-list. The out-list is a product, not a confession. “Custom themes, multi-device sync, and calls are parked.” If the interviewer pulls one back, they just told you the deep dive.

Prioritization (must have vs nice to have)

Must-have is what you will design in this session. Nice-to-have is named so the interviewer knows you heard it. A useful split for a messenger MVP: deliver 1:1 text with offline inbox and basic presence. Nice-to-have: large groups, receipts, reactions, calls. Must-haves get APIs and storage. Nice-to-haves get a one-line “how we would extend.”

Architecture Diagram

Requirements work produces a **decision tree**, not a data-center drawing. Only then does the default request path appear.

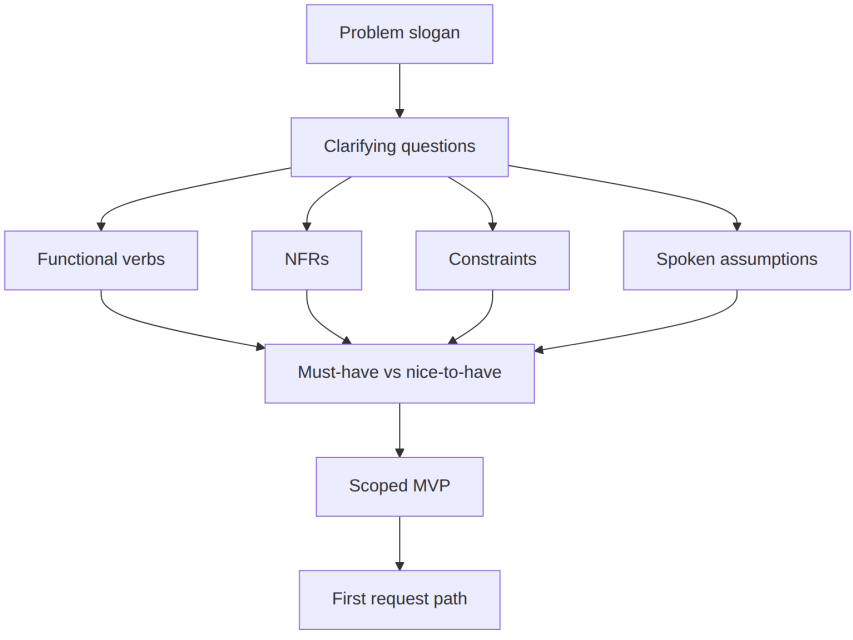


Figure 2.1 — Architects think left to right. Boxes come after Scope.

Once scope exists, the first path is still the simple one. You have not earned extra stores yet.

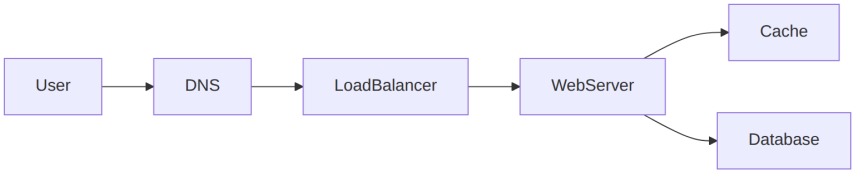


Figure 2.2 — After scope, a default path. Attach NFRs to hops: encryption at the client, latency at the edge, durability at the database.

Real-world Example

Prompt: “Design WhatsApp.”

A weak candidate draws WebSockets, Kafka, Cassandra, and a TURN server in two minutes. A strong candidate does this instead.

Clarifying questions (ask, then write the answers you agreed):

- Should it support one-to-one chat only?
- Do we need group chats? How large is a group in this round?
- Voice or video calls in this 45 minutes?
- Read receipts? Typing indicators?
- End-to-end encryption — required for MVP?
- Expected daily active users? Peak concurrent connections?
- Message retention period? Can the user delete everywhere?
- Multi-device? Last-seen privacy?

A defensible MVP for a 45-minute round:

Must have	Nice to have (parked)
1:1 text, online and offline	Groups above a small size
Delivery to other devices of the same user *or* a stated single-device assumption	Voice/video calls
Basic presence (online / last seen as a coarse flag)	Read receipts, reactions
Transport encryption; E2E if they said it is must-have	Global search of all history
Retention: 30 days on server *or* "until user deletes" — pick one and say it	Channels / status / payments

Functional list: send; receive while the other party is offline; list recent conversations; (optional) notify.

NFRs that change the diagram: delivery latency for online peers; durability of undelivered messages; if E2E is in, the server cannot be the search index for message bodies.

Constraints: mobile clients, flaky networks, a push-notification dependency you do not own.

Assumptions to say: “I will assume 100 million DAU unless you want internal-only scale. I will assume 1:1 first, groups as an extension of the same fan-out path.”

Only now do you sketch connections, a message store keyed by conversation, and a fan-out path. The boxes were earned.

This is how architects think before they design: the product is a set of verbs under limits, not a brand name.

Enterprise Insight

In an enterprise the same prompt is often “design our internal messenger” and the hidden requirements live in other documents: records-retention holds, eDiscovery, data residency, DLP, SSO, and “must run on the corporate device fleet.” Those are constraints, not features. A consumer-style E2E design may be **illegal** for a bank that must retrieve messages under discovery. Ask who the legal audience is. Interviewers for enterprise roles listen for that question. Interviewers for consumer roles listen for privacy. Same slogan, opposite NFRs.

Prioritization in a company is political: security wants E2E, compliance wants search, product wants receipts. Your job in the room is to name the conflict, not to pretend it is not there.

Interviewer’s Mind

They are scoring whether you **negotiate the problem**. A candidate who lists twenty features is scared of missing points. A candidate who asks about 1:1 versus groups and then parks calls is running the room.

Weak: jumping to CAP theorem or a vendor list before any user story.

Strong: “Which of delivery latency or on-device encryption should win if they conflict?” and “Must-have is 1:1 text with offline inbox.”

If they refuse to answer a clarifying question, they want you to assume — so assume **out loud**.

AI Perspective

If the messenger includes "smart replies," moderation, or translation, those are extra verbs with extra NFRs: a model on the hot path vs async, training-data residency, and whether E2E even allows a server-side model to see plaintext. Default: keep AI **off** the send path; run moderation on a copy only if legal and product allow it; fail open so chat still works when the model is down. Do not bolt a chatbot onto WhatsApp in minute two to sound current.

Common Mistakes

- Drawing boxes before a single clarifying question.
- Collecting NFRs like trophies ("highly available, strongly consistent, globally encrypted, cheap").
- Treating the brand name as the spec.
- Never writing non-goals.
- Designing calls, payments, and channels because the real app has them.

Best Practices

- Questions first, then a two-column must/nice table.
- 5–8 functional bullets, two circled NFRs, spoken assumptions.
- Repeat the interviewer's answers in your own words.
- Keep the parking lot visible.
- Revisit requirements after estimates (Chapter 3): numbers kill pretty NFRs.

Summary

Requirements turn a slogan into a system. Functional verbs say what. NFRs say what would change the picture. Constraints are non-negotiable. Assumptions are spoken. Clarifying questions

are how architects think. Scope and prioritization are how you finish on time. For “design WhatsApp,” the win is the questions and the MVP table, not a TURN server in minute one.

Key Takeaways

- Interviews start with requirements, even when the prompt is a product name.
- Must-have versus nice-to-have is a design artifact.
- Clarifying questions should change boxes, SLOs, or data models.
- Enterprise and consumer messengers can be opposite designs under the same slogan.
- Do not draw until scope exists.

Interview Questions

- Walk through the first five questions you would ask for “design WhatsApp.”
- Give an NFR that would split one chat service into two.
- How do you handle an interviewer who keeps adding features at minute 30?
- When is end-to-end encryption a must-have versus a parked item?
- What is the difference between a constraint and an assumption on the board?

Further Reading

- Chapter 3, where this MVP becomes users, QPS, storage, bandwidth, and memory.
- Chapter 10, where two NFRs become an explicit trade-off sentence.
- Appendix A, steps 1–2.
- Appendix A, steps 1–2.
- Your organization’s architecture intake form: strip it to what still fits in ten minutes.

Capacity estimation

Learning Objectives

- Estimate from a scoped MVP: users, QPS, storage, bandwidth, memory, and growth.
- Convert units with powers of two and attach latency orders of magnitude to hops.
- Show arithmetic that is honest to an order of magnitude.
- Attach each number to a hop or a component.
- Know when a number is good enough to stop.

Quote

> *A cache is a claim about volume. If the volume is not on the board, the cache is a superstition.*

Introduction

Back-of-the-envelope math is not a performance exam. It is how you stop guessing after Chapter 2. Interviewers watch whether "100 million users" becomes QPS, then bytes on the wire, then RAM for the working set, then a growth line that would force a partition next year.

You will be wrong by 2–3×. That is fine. Being wrong by 100× because you never multiplied is not. This chapter is the conversion table: **users, QPS, storage, bandwidth, memory, growth**.

Core Concepts

Users

Start from the assumption you spoke in Chapter 2. Daily active users (DAU) are not concurrent users. Concurrent is closer to "how many hold a connection or hit an API in the same second."

A rough interview move: peak concurrent ≈ a small fraction of DAU (you might start at 1–10% and ask them to correct you). Write both numbers.

QPS

Queries per second (or messages per second) come from concurrent users × actions per user per second, or from DAU × actions per day / 86,400, then a peak factor (often 2–5× average). Split **read QPS and write QPS**. A messenger is write-heavy on send and read-heavy on sync-after-offline. A URL shortener is the opposite.

Storage

Storage is records × size × retention × replicas. Message bodies, attachments, and indexes are different lines. Five years of click logs is not the same as five years of 200-byte mappings. Say replication factor (3× is a common first guess) so you do not "fit on one disk" by forgetting copies.

Bandwidth

Bandwidth is QPS × payload size, inbound and outbound. Fan-out multiplies outbound: one send to a group of 50 is 50 deliveries. Video is a different era of numbers; if calls are parked, do not mix them into text bandwidth.

Memory

Memory is for **working sets** you want hot: open sessions, recent conversations, cache keys. RAM is not "the database." Estimate: active sessions × session size, plus cache working set. If the hot mapping table is 40 GB and a box has 64 GB, a cache tier is plausible. If it is 40 TB, you are telling a partition story, not a Redis story.

Growth projections

Ask “same design at 2× and 10× in 18 months?” Growth is users, QPS, and stored bytes — they do not grow at the same rate. Storage often grows even if DAU is flat (retention). QPS grows with engagement. Say which lever you would pull first at 10× (Chapter 4) instead of redrawing everything now.

Powers of two (unit conversion)

Keep a tiny conversion table in your head so storage estimates do not stall:

Approx	Meaning on the board
2^10	thousand, ~1 KB
2^20	million, ~1 MB
2^30	billion, ~1 GB
2^40	trillion, ~1 TB

A 200-byte message × 10^8 users is not “a lot of data” until you multiply by messages per day, retention, and replicas. Powers of two turn that multiplication into GB/TB without a calculator.

Latency orders of magnitude

Capacity is not only bytes. It is whether an operation fits the **time** budget (Chapter 7). Orders of magnitude you should be able to say without precision theater:

- In-process / RAM: tens to hundreds of nanoseconds.
- Same-datacenter round trip: on the order of half a millisecond.
- SSD random read: hundreds of microseconds.
- Disk seek: around ten milliseconds.
- Continent-to-continent: around a hundred milliseconds or more.

Handy consequences: you get a couple of thousand round trips per second inside one DC, and only a handful of transatlantic round trips per second. Do not put N disk seeks on a redirect path. Do not pretend a cross-region consensus round will meet a 50 ms p99.

Stop condition. Once a number has justified a component or killed one, move on.

Architecture Diagram

Estimates attach to hops.

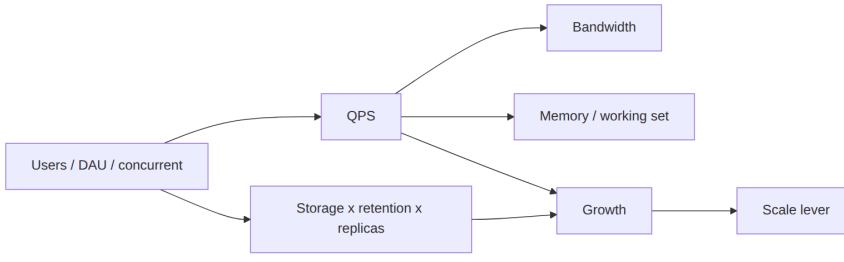


Figure 3.1 — Six quantities, one lever. Do not skip from Users to Kafka.

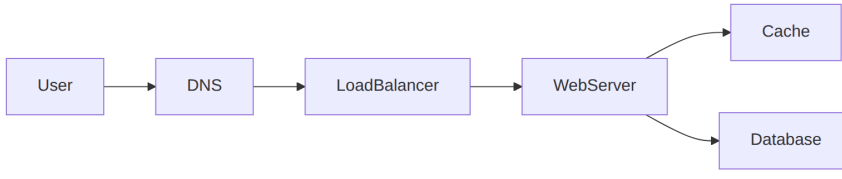


Figure 3.2 — Where the numbers land: connections and QPS on the web tier; working set on the cache; retained bytes on the database and object store.

Real-world Example

Take the WhatsApp-like MVP from Chapter 2, with a spoken assumption of **100 million DAU**, 1:1 text only, 40 messages per user per day, 200-byte payload, 30-day server retention of undelivered-plus-recent, attachments parked.

- **Users:** 100M DAU. Peak concurrent connections: assume 5M (5%) unless they correct you.

- **QPS:** 100M × 40 / 86,400 ≈ 46k messages/s average. Peak ×3 ≈ 140k/s. That is send+receive; count both if each is an API.
- **Storage:** 100M × 40 × 200 B × 30 days ≈ 2.4 TB/month of raw text before indexes and replicas. ×3 replicas ≈ 7 TB class — uncomfortable on one primary, comfortable as a cluster. Attachments would dominate if you un-parked them.
- **Bandwidth:** 140k/s × 200 B ≈ 28 MB/s payload at peak for one hop — small for text, irrelevant compared with fan-out or media.
- **Memory:** 5M connections × 2 KB connection state ≈ 10 GB just for sockets/session — that already suggests more than one connection box. Cache of recent threads is a separate working-set line.
- **Growth:** 2× DAU in a year doubles QPS and connection RAM; storage also grows with retention policy. At 10× you partition by conversation ID (Chapter 4), you do not add a new product family in a panic.

A different assumption — **50,000 enterprise users** — yields tens of QPS. Then a single primary is honest, and talking about regional Kafka is theater. The method is the same; the numbers decide the boxes.

Enterprise Insight

Enterprises already have capacity: last year's gateway QPS, warehouse size, backup window. In a real review you start from measured data. In an interview you invent a baseline and offer it for correction. Either way, **cost is an NFR hiding inside capacity**. Connection-heavy designs (long-lived sockets) cost RAM and file descriptors, not only CPU. Retention policies from legal can dwarf engineering's "keep messages forever" instinct — 7-year mail archives are a storage product, not a chat feature.

Interviewer's Mind

They are not checking your mental arithmetic against an answer key. They are checking whether you **use** the number. Weak: "assume high QPS" then draw the same diagram you always draw. Strong: "writes are tens of thousands per second, payloads are tiny, working set of connections is the first bottleneck — I will scale the connection layer before I shard the message store."

If they give you a number, treat it as sacred until they change it.

AI Perspective

Inference changes the table. Tokens in and out have cost and latency that dwarf a 200-byte message. Batch embeddings off the hot path. If you add on-device or server-side smart replies, estimate QPS of inference separately from message QPS; GPU memory is a different line from connection RAM. Do not fold "AI" into the 28 MB/s text bandwidth and call it done.

Common Mistakes

- Treating DAU as concurrent.
- Precision theater (17,342.7 QPS) with no component decision.
- Forgetting replicas, indexes, and logs in storage.
- Mixing parked video calls into the text bandwidth line.
- Skipping memory for connection-oriented systems.

Best Practices

- Fill the six-line table: users, QPS, storage, bandwidth, memory, growth.
- Write units. Convert to per-second once, then round.
- Split read vs write, payload vs attachment.
- Announce the decision the number supports.
- Invite the interviewer to correct the baseline.

Summary

Capacity estimation is a small table that earns the next box. Users become QPS; QPS and payload become bandwidth; retention becomes storage; working set becomes memory; time becomes growth. Show the work, round aggressively, and stop when the architecture has a reason.

Key Takeaways

- Six quantities: users, QPS, storage, bandwidth, memory, growth.
- DAU is not concurrency.
- Components must be justified by volume or they are decoration.
- Enterprise scale and consumer scale use the same table and different boxes.
- Being roughly right beats being silently wrong.

Interview Questions

- Convert 100 million DAU and 40 messages/user/day into average and peak QPS.
- How would you estimate storage for 30-day text versus 5-year attachments?
- Why might connection memory saturate before database disk?
- What growth line would you watch if DAU is flat but legal extends retention?
- When do you stop estimating in a 45-minute interview?

Further Reading

- Chapter 4, where these numbers become scale-out stories.
- Chapter 7, where QPS meets latency budgets.
- Appendix A, step 3, plus the powers-of-two and latency-order notes.

- Public latency-order references (widely taught large-scale systems talks): use orders of magnitude; do not paste tables.

Scalability

Learning Objectives

- Distinguish a performance problem (slow for one user) from a scalability problem (fine for one user, slow under load).
- Distinguish vertical scale, horizontal scale, and work you should not do on the hot path.
- Choose partition keys and say what happens when they are hot.
- Scale reads, writes, and storage as separate problems.
- Describe what breaks at 10× without redrawing the whole board.

Quote

> *Scale is not a property of Kubernetes. It is a property of the bottleneck you have not named yet.*

Introduction

"Make it scalable" is how slogans sneak back in after you just finished requirements. Scalability means: when load grows, which resource saturates first, and what lever do you pull? Compute? Disk? A single lock? A single customer ID? This chapter is that lever, in first principles.

Core Concepts

Performance versus scalability. If one user already waits, you have a **performance** problem (Chapter 7): fix the path, the query, the p99 hop. If one user is fine and the fleet collapses at peak, you have a **scalability** problem: the next unit of load needs more machines, shards, or queues. A design is "scalable" only if added resources buy added work in proportion. A hot key violates that: ten more boxes do not help.

Vertical scaling is a bigger machine. It is honest, fast to ship, and finite. Mention it. Senior people who skip it look like they have never been on call at 2 a.m. with a budget freeze.

Horizontal scaling is more machines doing the same kind of work. It requires **stateless app tiers** and **partitioned data**. If session state lives in the app process, you do not have horizontal scale. You have sticky routing and a future incident.

Partitioning splits data by a key. The key must spread load. User ID often works; a boolean "is_active" never does. Hot keys (a celebrity, a flash sale SKU) need a special story: cache, split, or isolate. Placement itself must not depend on the live fleet size: `hash(key) % N` remaps almost every key when a box dies. Consistent hashing (Chapter 14) is the interview name for keeping most keys put.

Scale the right axis. Reads like caches and replicas. Writes like queues, log-structured stores, and careful primary design. Storage like object stores and tiering. Mixing those sentences is how diagrams become mush.

Architecture Diagram

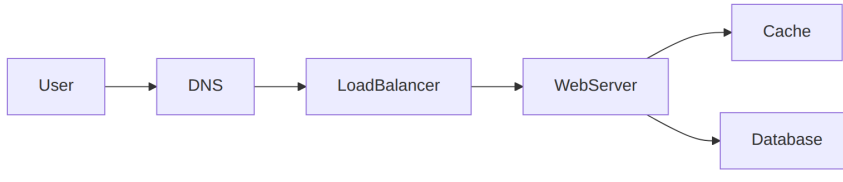


Figure 4.1 — Where scale levers sit: DNS and the load balancer spread connections; web servers scale out if stateless; cache absorbs repeated reads; the database scales only after you name a partition key or a replica role.

Add a second web server only after you say "stateless." Add a second database only after you say "replica for reads" or "shard by user_id."

Real-world Example

A checkout service dies every holiday because inventory for one SKU is a single row. More pods do nothing; they serialize on the same row. The scale fix is not “Kubernetes HPA.” It is inventory as a reservation log, or sharded stock buckets, or a queue in front of a single serializer with honest backpressure. The interview version: name the hot key before you draw a cluster.

Enterprise Insight

Platform teams will offer you autoscaling groups and managed databases. Take them. Your job is still the **data plane story**: what is replicated, what is sharded, what is a cache that can vanish. Enterprises also scale *organizations*. A service that needs a cross-team change to add a partition is not horizontally scalable in practice. Prefer designs a single team can operate at 10×.

Interviewer’s Mind

They wait for the sentence: “At 10×, this primary becomes the bottleneck; I would partition by X; the risk is Y.” Candidates who chant “microservices” without a bottleneck get a follow-up that feels like an ambush. It is not an ambush. You never named the resource.

AI Perspective

Model serving scales with batch size, GPU memory, and queue depth, not with the same curve as a JSON API. Feature stores and embedding indexes are additional data systems with their own partition keys. If AI ranking is on the read path, 10× users may 10× GPU cost before they 10× your database. Call that out; it is a more mature answer than “we will use a bigger model.”

Common Mistakes

- Equating “more services” with “more scale.”
- Sharding before a single primary is actually full.

- Ignoring hot keys.
- Scaling the app tier while the database connection pool is the ceiling.

Best Practices

- Say “stateless” before you duplicate app boxes.
- Name the partition key and one hot-key mitigation.
- Separate read scale from write scale on the board.
- Describe the 10× story in two sentences at the end of the design.

Summary

Scalability is the named bottleneck and the lever you will pull. Stateless compute, careful keys, and different tactics for reads versus writes beat a generic cluster drawing.

Key Takeaways

- Vertical scale is a valid first lever.
- Horizontal scale needs stateless apps and a partition story.
- Hot keys are a scale problem, not an edge case.
- 10× should change one part of the diagram, not all of it.

Interview Questions

- When would you refuse to shard in the first 45 minutes?
- How do you scale reads without scaling writes?
- What makes a bad partition key?

Further Reading

- Chapter 7 for the single-user performance side of the diagnostic.
- Chapter 5, availability, which is often in tension with scale-out writes.
- Chapter 12, storage, for replica and shard mechanics.

- Chapter 14, consistent hashing, when N of the cache or shard pool is not eternal.
- Chapter 6, reliability, when scale-out creates more failure domains.
- A postmortem from your own hot-key incident, rewritten as a board story.

Availability

Learning Objectives

- Define availability as an SLO, not as “the site is up.”
- Translate nines into downtime and combine components in series versus in parallel.
- Contrast active-passive and active-active failover.
- Place redundancy on the path: instances, zones, and (when asked) regions.
- Pair retries with idempotency and timeouts.
- Explain when a degraded mode is better than a full outage.

Quote

> *Three nines is a budget. You spend it on change, on dependencies, and on the one database you made a single point of failure.*

Introduction

Availability is the share of time the user can complete the verb you promised. It is not “we run Kubernetes.” It is redundancy, failure detection, and what the product does when a dependency is sick. Interviews use it to see whether you design for **failure as a normal input**.

Core Concepts

SLO. Pick a number you can defend (for example, 99.9% for an internal tool, tighter for checkout). The remaining fraction is your error budget for deploys and incidents.

Redundancy. Two app instances behind a load balancer survive one crash. Two zones survive one data center. Two regions survive a disaster and create a **data** problem: what is replicated, how far behind, who is primary.

Timeouts, retries, backoff. A retry without a timeout is a herd. A retry without idempotency is a double charge. Draw them on the client-to-edge hop and on the app-to-database hop as policies, not as hope.

Degradation. If the recommendation service is down, show a cached or unranked list. If payments are down, do not pretend the catalog is the problem. Partial is a feature.

Nines. Availability is often spoken as nines of uptime. Rough downtime if the year were the only window: three nines (~99.9%) is on the order of a workday per year; four nines (~99.99%) is under an hour per year. The useful interview move is not memorizing seconds — it is noticing that a weekly deploy window or a 30-minute failover already **spends** three nines. Measure the SLO on the user verb, not on "the VM pinged."

Series versus parallel. Two dependencies on the same request path multiply. Two boxes each at 99.9% in **series** are about 99.8% together. Two redundant boxes in **parallel** (either one can serve) push combined unavailability down sharply — if failover actually works. A licensed identity appliance in one AZ is a series component; extra app replicas do not save you.

Failover shapes. Active-passive: a standby takes over (hot standby is already running; cold standby must boot — that time is downtime). **Active-active:** both sides take traffic; clients or DNS must know both. Cost: more hardware and a split-brain or lost-write risk if the active dies before replication (Chapter 6).

Architecture Diagram

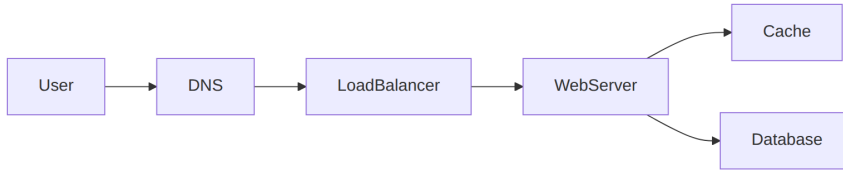


Figure 5.1 — Availability levers: DNS can fail away from a sick region; the load balancer hides dead web servers; cache can serve stale reads if you accept it; the database needs a replica or a failover story or it is the single point of failure.

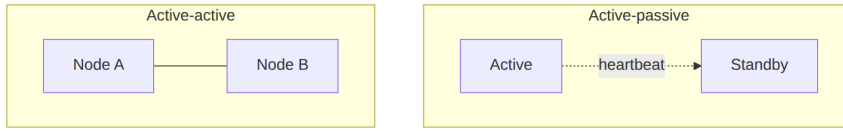


Figure 5.2 — Standby waits; active-active shares load. Name data replication or you only drew compute HA.

Real-world Example

A ticket site stays “up” (the homepage 200s) while checkout hangs on a payment provider for 30 seconds with no timeout. Users retry, doubling load. Availability of the *homepage* was fine. Availability of **purchase** was zero. Design the SLO around the verb that makes money, and put a timeout plus a queue or a fail-fast on that hop.

Enterprise Insight

Enterprises buy multi-AZ defaults from the cloud and then put a license server or an identity appliance in one zone. Draw **your** single points of failure, including the ones facilities and vendors own. That appliance sits in **series** with everything else and wrecks the nines you wrote on the slide. Change management eats error budgets; a design that requires a weekend outage to migrate shards has an availability cost even if the runtime graph looks redundant. Mention operable failover: who flips the DNS, how you avoid split brain, how you test it.

Interviewer’s Mind

They listen for “single point of failure” in your own diagram, said by you first. They like “we will serve stale cache if the DB is down for reads that allow it.” They dislike “we are highly available” with one box labeled “DB.” Multi-region is an advanced move; do not volunteer it until the numbers or the interviewer demand it.

AI Perspective

Model APIs are flaky compared to a well-run cache. If generation is required for the user-visible verb, you need a fallback: template, cached answer, or a smaller on-platform model. If generation is decorative, **fail open**. Availability of an AI feature should not take down search or checkout. Rate-limit inference separately so a prompt storm cannot exhaust the error budget of the core API.

Common Mistakes

- Equating replication with availability (replicas lag; failovers split brains).
- Infinite retries.
- Multi-region as the first drawing, with no data story.
- Health checks that only prove the process is alive, not that it can do work.

Best Practices

- SLO on the user verb.
- Timeout plus bounded retry plus idempotency keys on writes.
- Name the failover: who becomes primary, how clients find it.
- Design one degraded mode.

Summary

Availability is an error budget, redundancy on the path, and behavior under dependency failure. The database and the unpaid vendor are the usual single points. Retries without idempotency turn outages into corruption.

Key Takeaways

- SLO first, adjectives second.
- Redundancy without failover practice is a drawing.

- Retries need timeouts and idempotency.
- Degrade the accessory, protect the verb.

Interview Questions

- How does a read replica help availability, and how does it not?
- What do you do when a downstream payment API is at 30% errors?
- When is multi-region the wrong availability move for a 45-minute interview?

Further Reading

- Chapter 6, reliability, for retries and idempotency in depth.
- Chapter 13, where async delivery changes what “available” means.
- Chapter 15, putting availability into the interview loop.
- Your last incident timeline: map each minute to a missing timeout or a missing replica.

Reliability

Learning Objectives

- Define reliability as “correct work over time,” distinct from availability.
- Place timeouts, retries, idempotency, and backoff on the path.
- Use health checks that prove work, not only liveness.
- Describe poison messages, at-least-once delivery, and degraded correctness.

Quote

> *Availability is whether the door opens. Reliability is whether the room you entered is the one you were promised.*

Introduction

Chapter 5 asked whether the service is up. This chapter asks whether it **does the right thing** while it is up: no double charges, no dropped messages that the client thought were sent, no retries that amplify an outage. Interviewers use reliability to see if you have been on call.

Core Concepts

Reliability is the probability that a unit of work completes correctly inside its contract. A site can be “available” (HTTP 200) and unreliable (wrong inventory, missing messages).

Timeouts bound waiting. **Retries** repeat work. Together they need **idempotency**: the same write twice must not create two orders. **Backoff and jitter** stop retry storms.

Health: liveness (process exists) vs readiness (this instance can do work). Load balancers should use readiness.

Failover is not reliability. Active-passive (Chapter 5) can still lose the last write if the active dies before the standby has it. That is a **correctness** hole, not an "uptime" hole. Replication lag is a reliability input.

Poison work: a payload that always fails. It must dead-letter, not block the queue forever.

Architecture Diagram

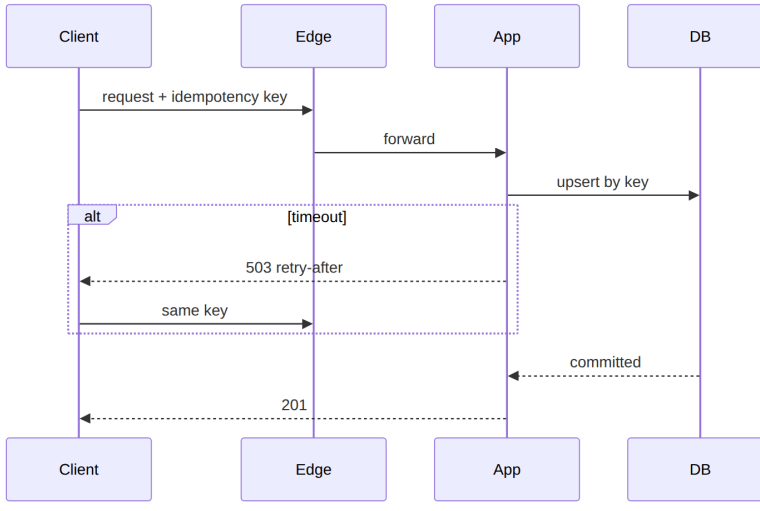


Figure 6.1 — Reliability is a path with a key, a timeout, and a safe retry. The database enforces "once" for that key.

Real-world Example

A payments API returns 504. The client retries and the customer is charged twice because the first write actually landed. Reliability fix: idempotency key from the client, unique constraint, and a timeout shorter than the user's patience. Availability of the API was "mostly up." Reliability of "charge once" was broken.

Enterprise Insight

Enterprises care about audit: you must show that a retry did not duplicate a legally relevant event. That pushes you toward an outbox and stored idempotency keys with a retention that matches finance, not a 5-minute cache. SLOs for reliability (failed orders, duplicate sends) sit beside availability SLOs. Change management is a reliability input: most incorrect work is a bad deploy, not a cosmic ray.

Interviewer's Mind

They wait for "idempotency key" on writes and "what if the worker dies after send but before ack." They dislike infinite retries. They like a dead-letter with an owner. Do not say "exactly once" unless you can explain the keys.

AI Perspective

Model calls fail more often than disk writes. Treat inference as an unreliable dependency: timeout, fallback (cached or template), and never retry a non-idempotent "bill this prompt" without a key. Duplicate completions can leak data or cost; they are a reliability bug, not a quirk of AI.

Common Mistakes

- Retries without keys.
- Health checks that only ping localhost.
- Equating replication with correctness.
- Swallowing errors to keep availability green.

Best Practices

- Idempotency on every user-visible write.
- Bounded retries with jitter.
- Readiness vs liveness.
- Dead-letter plus metric.

Summary

Reliability is correct work under failure. Timeouts, retries, and keys are the mechanism. Availability without reliability is a green dashboard and an angry finance team.

Key Takeaways

- Up is not correct.
- Retry only what is safe to repeat.
- Health must mean “can do work.”
- Poison messages need an exit.

Interview Questions

- How do you make “send message” safe to retry?
- What does a 200 with the wrong body do to your SLO?
- When should a limiter fail open versus fail closed (reliability vs availability)?

Further Reading

- Chapter 5 for the availability pairing.
- Chapter 13 for at-least-once consumers.
- Chapter 8 for when retries conflict with consistency.
- Failover data-loss as a reliability topic in community availability notes — describe it on your own path.

Performance

Learning Objectives

- Set latency as a percentile SLO (p50/p99), not “fast.”
- Separate latency from throughput and say the tension (batching).
- Find the hop that dominates p99.
- Use queues and caches as performance tools only with a stated cost to freshness.

Quote

> *Average latency is a compliment you pay yourself. Users live in the tail.*

Introduction

Performance in interviews is not micro-benchmarks. It is whether the verb from Chapter 2 completes inside a budget derived from Chapter 3. p99 is where locks, GC, cold caches, and noisy neighbors show up. The other axis is **how many** of those verbs you complete per second. This chapter is latency versus throughput, then tails.

Core Concepts

Latency versus throughput

Latency is the time to finish **one** action (one redirect, one send). **Throughput** is how many such actions you finish **per unit time**. They are not the same knob. A pipeline can move a million tiny redirects per second (throughput) while one unlucky user still waits 800 ms (latency).

A useful default: **raise throughput until latency is no longer acceptable**, then stop. Batching is the classic tension — larger batches usually raise throughput and raise wait time for the first

item in the batch. A URL shortener wants tiny latency on GET; a log shipper wants throughput. Do not optimize one with the other's metric.

Percentiles. p50 is typical. p99 is what you design for if the user is waiting. Tail amplification: one slow dependency in serial ruins p99. Throughput-only dashboards hide that.

Budgets. If the page must be 200 ms, and you have DNS, TLS, app, cache, DB, something gets 10–20 ms. Serial hops add; parallel hops need a join.

Utilization. A disk at 90% has ugly tails. Headroom is a performance feature.

Architecture Diagram

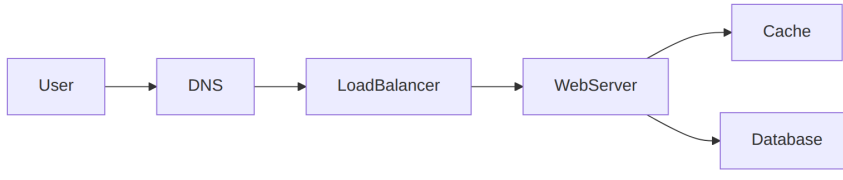


Figure 7.1 — Budget the path. Cache exists to cut database time on hits; it can hurt p99 on stampedes (Chapter 11).

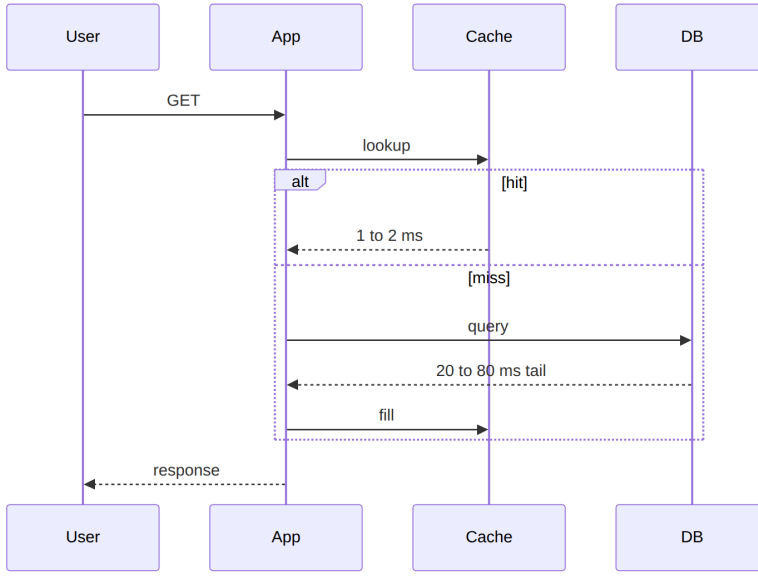


Figure 7.2 — The miss path is the p99 you must name.

Real-world Example

Redirect of a short link: budget 50–100 ms p99 worldwide. Most of that is distance, not JSON (Chapter 3 latency orders). A cache of mappings makes the app hop boring. Putting click analytics on the same round-trip makes p99 a function of a warehouse. Performance design is **where you refuse to wait**. Throughput of creates is a different budget — rate-limit that (Chapter 17) instead of slowing GET.

Enterprise Insight

Enterprises have WAN latency, proxies, and DLP scanners on the path. Your beautiful 20 ms service becomes 200 ms in a branch office. Design p99 for the real client, or offer a regional edge. Capacity and performance reviews often fight: finance wants 90% utilization; tails want 50–70% on stateful stores. Say that trade-off (Chapter 10).

Interviewer's Mind

They like "p99" and a single bottleneck hop. They dislike "we will optimize later" with no budget. They may ask how you would measure; name a histogram, not an average.

AI Perspective

Token generation is streaming latency, not a single p99 of a JSON blob. Time-to-first-token and tokens/sec are the metrics. Do not put a 2-second model in front of a 100 ms checkout. Batch and cache embeddings so search p99 is index time, not GPU time.

Common Mistakes

- Optimizing averages.
- Adding hops to "help performance."
- No budget before a cache.

- Synchronous fan-out to many services on the user path.

Best Practices

- Write p99 on the board.
- Count serial hops.
- Keep the hot path short; async the rest.
- Leave headroom on stateful tiers.

Summary

Performance is a percentile budget on a named path. Throughput and utilization are cousins. The miss path and the slowest serial dependency own the tail.

Key Takeaways

- Users live in p99.
- Latency and throughput are different knobs; batching trades one for the other.
- Every extra hop is a bet against the tail.
- Headroom is part of the design.

Interview Questions

- Why can p99 get worse when you add a cache?
- How do you budget a 200 ms page with three serial dependencies?
- What utilization would you refuse on a primary database?

Further Reading

- Chapter 3 for QPS, payload, and latency orders of magnitude.
- Chapter 4 for “slow under load” versus “slow for one user.”
- Chapter 11 for cache tails.
- Chapter 10 for performance versus cost.

CAP theorem

Learning Objectives

- State CAP in operational language: when a partition happens, you choose consistency or availability for that decision.
- Stop using CAP as a product label ("we are CP").
- Tie the choice to a user verb from Chapter 2.
- Know what CAP does *not* say (latency, PACELC, "2 of 3 in normal times").

Quote

> *CAP is not a personality type for your database. It is a fork in the road when the network lies to you.*

Introduction

CAP shows up in interviews as a trap: candidates recite "consistency, availability, partition tolerance — pick two" and then pick a vendor. Architects use a narrower statement: **if the network between replicas is broken, do you refuse the write (preserve a single picture of the data) or accept the write (stay up, risk divergence)?** Partition tolerance is not optional in a distributed system; the interesting choice is the other axis.

This chapter gives you language that survives a follow-up. Chapter 9 names the consistency models you actually ship. Chapter 10 turns the fork into a trade-off sentence.

Core Concepts

Partition. A replica cannot get timely, correct answers from another replica. Timeouts are how partitions look in real life.

CP-style choice (for this decision): reject or delay the operation until you can agree. The user sees an error or a wait. The data model stays single-copy in intent.

AP-style choice (for this decision): proceed with local state. The user sees success. Copies may disagree until repair.

Interview wording of the three promises (say them, then apply to a verb):

- **Consistency:** a read returns the latest completed write, or an error.
- **Availability:** a request gets *some* timely answer; it may be stale.
- **Partition tolerance:** the system still attempts to serve when the network between copies is sick.

Networks fail, so you do not get to drop partition tolerance in a distributed design. The software choice is consistency versus availability **for this verb** when a timeout looks like a partition. Waiting on the isolated copy is the CP-shaped move (good when the business needs atomic reads/writes). Serving the local copy and repairing later is the AP-shaped move (good when eventual visibility is allowed).

What CAP is not. It is not “we never have latency.” PACELC reminds you: even without a partition, you still trade latency vs consistency. It is not “SQL is CP and NoSQL is AP.” Those slogans fail in the room.

Architecture Diagram

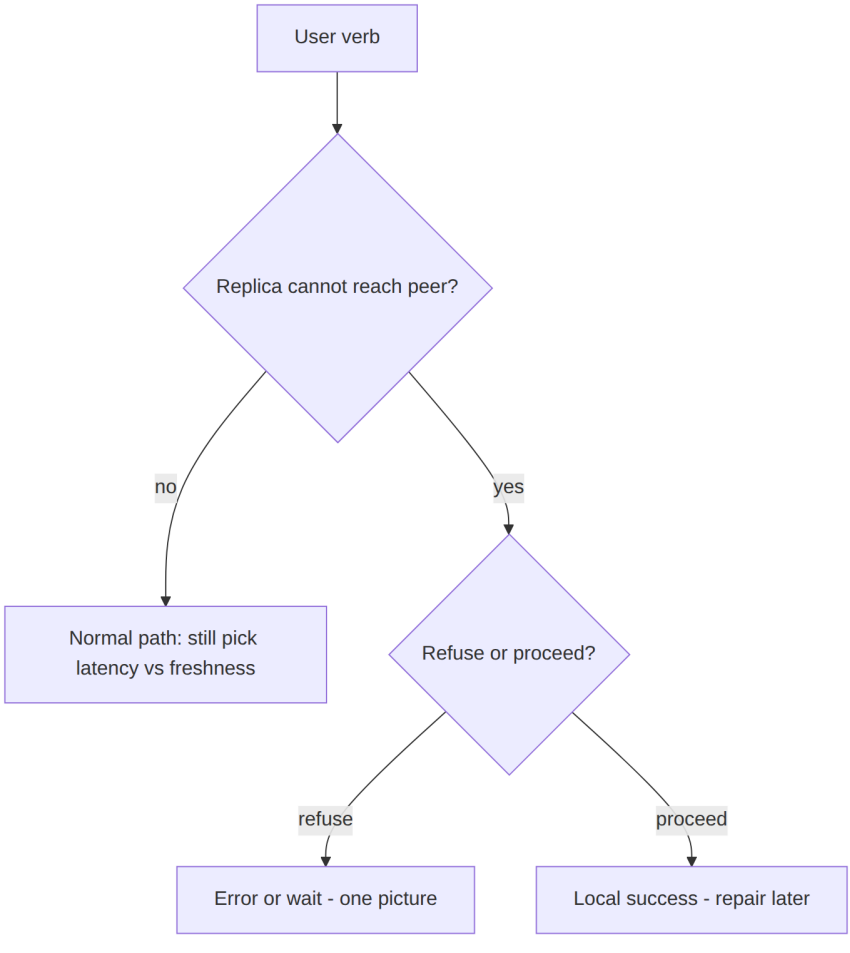


Figure 8.1 — CAP as an operational fork, not a logo on the database cylinder.

Real-world Example

Messenger **send** with a requirement “do not lose the message”: on partition between the client’s home store and a replica, you may still accept the write in the home region and repair (availability of send, delayed global visibility). Messenger **group admin** “only one owner”: you may refuse the second promote during a partition. Same product, two forks. The clarifying questions from Chapter 2 told you which verb you are in.

Enterprise Insight

Enterprises often pick CP on money and identity, AP on telemetry and presence. The network partitions they actually meet are AZ blips, not textbook datacenter splits — timeouts still fire. Multi-region active-active is an AP-shaped offer unless you have a consensus round on the path (and then you bought latency). Legal “one global balance” is a CP demand; product “never show an error” is an AP demand. Name the conflict.

Interviewer’s Mind

They want you to **apply** CAP to a verb, not define it from a slide. Weak: “Cassandra is AP so we are fine.” Strong: “During a partition I will still take chat messages in the local region and reconcile; I will not take a duplicate payment.” If you say “pick two,” they may ask “which two while the network is healthy?” — have the PACELC sentence ready.

AI Perspective

Feature stores and prompt caches are usually AP-shaped: stale features beat down search. Ledger of token billing is CP-shaped. Do not run billing counters in the same consistency mode as a recommendation cache.

Common Mistakes

- Labeling a whole company “AP” or “CP.”
- Forgetting that partitions look like timeouts.
- Using CAP to avoid drawing a failover.
- Claiming a system is CA in the CAP sense while it is distributed.

Best Practices

- Name the verb, then the fork.
- Admit PACELC: latency vs consistency in the happy path.
- Do not tattoo CAP on a vendor.

- Connect to Chapter 6: retries during a partition can duplicate AP writes.

Summary

CAP is the choice you make when replicas cannot agree in time: refuse (one picture) or proceed (stay up, repair). Apply it per operation. Leave slogans off the board.

Key Takeaways

- Partitions are timeouts between copies.
- Choose per verb, not per resume bullet.
- Healthy-path latency is a different trade-off (PACELC).
- Vendor labels are not a CAP proof.

Interview Questions

- For a wallet debit, what do you do if the replica in another AZ does not answer in 100 ms?
- How can one product be both CP-like and AP-like?
- What does CAP not tell you about p99 latency?

Further Reading

- Chapter 9, consistency models you can actually name on the board.
- Chapter 5, availability as the other side of the fork.
- Chapter 10, turning this fork into a spoken trade-off.

Consistency models

Learning Objectives

- Name linearizability, sequential consistency, causal consistency, and eventual consistency in user-visible terms.
- Map read-your-writes, monotonic reads, and bounded staleness to client experience.
- Choose a model for a verb without hiding behind a database logo.
- Connect replica lag to what the user is allowed to see.

Quote

> *Consistency is not a feeling of safety. It is a promise about which writes a read is allowed to miss.*

Introduction

Chapter 8 was the emergency fork. This chapter is the everyday promise: after a write, who sees what, and how soon? Interviewers want words you can test. “Strong” and “weak” are not enough.

You do not need a graduate seminar. You need four or five models and a client-centric checklist.

Core Concepts

Linearizability. There is a single real-time order. After a write completes, every reader sees it. Expensive across regions.

Sequential consistency. Operations from each client appear in order, and there is one global order, but it need not match wall-clock. Rarer in interviews; mention only if they push.

Causal consistency. If A happened-before B (you saw A, then wrote B), nobody sees B without A. Comment threads care about this.

Eventual consistency. If writes stop, replicas converge. In the meantime, reads may be stale. You must say how stale is OK.

Weak consistency (no promise). After a write, a read might never see it. Live media and some caches behave this way: if a call drops for two seconds, you do not replay the lost audio. That is not "eventual" — eventual still owes you convergence.

Informal labels vs this book. Interviews often use only weak / eventual / strong. Map them so you are not arguing about names:

Informal label	In this book	Typical verb
Weak	No promise the write is seen	Presence pulses, live packets
Eventual	Converges if writes stop	DNS, like counts
Strong	Completed write is visible to later reads	Balances, unique short codes

"Strong" in that informal cut is closest to synchronous replicate / linearizable reads — still say *which*.

Client-centric extras:

- **Read-your-writes:** the writer sees their own write.
- **Monotonic reads:** you do not go backward in time on refresh.
- **Bounded staleness:** "no older than T seconds."

Read replicas are a consistency choice: they buy scale (Chapter 4) and spend freshness (Chapter 7 tails and Chapter 8 forks).

Architecture Diagram

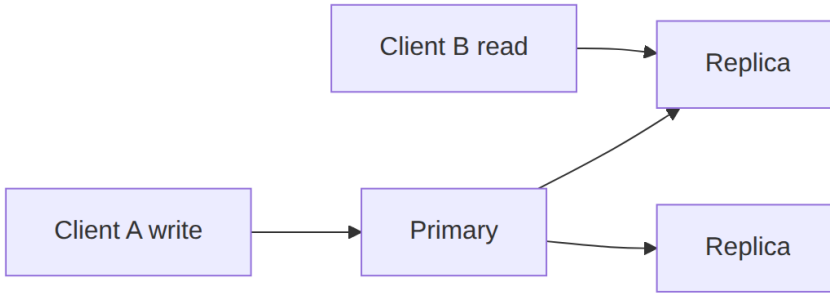


Figure 9.1 — If Client B hits a replica, the consistency model is "whatever lag you have not described yet." Name it.

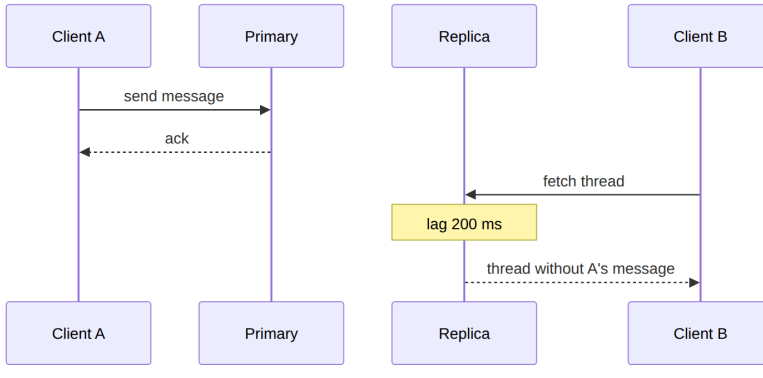


Figure 9.2 — Eventual / lagged read. If the product needs read-your-writes, A must read P (or a session that waits).

Real-world Example

WhatsApp-like 1:1 chat: the sender should read-their-writes on the device that sent (local first). Another device of the same user should catch up without reordering the thread (causal / session). A replica that shows a reply without the original message is a product bug, not a clever AP win. Presence "last seen" can be eventually consistent with seconds of lag. Do not give both verbs the same model.

Enterprise Insight

Reporting warehouses are eventually consistent by hours. Operational dashboards that drive trading cannot be. Enterprises often accidentally use a replica for a “did the payment succeed?” UI. Call that out. Session stickiness to a primary is an old trick to fake read-your-writes without linearizability across the fleet — and it fights scale. Multi-region “read local” is a consistency product decision sold as a latency win.

Interviewer’s Mind

They like “read-your-writes for the poster; eventual for the like count.” They dislike “we are strongly consistent” with a three-region active-active sketch and no consensus round. Ask which client must see the write before you pick the store.

AI Perspective

RAG indexes are eventually consistent projections of documents. A user who uploaded a PDF and immediately asked a question needs read-your-writes on that document path (wait for index, or search the source). Billing of tokens should not be eventual across the month-end report without a reconciliation job you can name.

Common Mistakes

- One model for the whole system.
- Ignoring replica lag when you add read replicas.
- Using “eventual” to mean “we did not think.”
- Promising linearizability across continents with a straight face and a 50 ms budget.

Best Practices

- Pick a model per verb.
- If you use replicas, say lag and session behavior.

- Prefer read-your-writes for the user's own writes.
- Convergence is not optional for eventual: name repair (async, anti-entropy, or "last write wins" and its damage).

Summary

Consistency models are promises about missed writes. Linearizability is the strict real-time promise. Eventual is a lag budget. Causal and session guarantees sit in between and match how humans read threads. Choose per verb.

Key Takeaways

- "Strong" is not a model.
- Read-your-writes is the most common client need.
- Replicas spend consistency to buy scale.
- Eventual needs a repair story.

Interview Questions

- How do you give read-your-writes without linearizable global reads?
- What breaks in a comment thread under last-write-wins eventual consistency?
- Why might you refuse a read replica for a balance query?

Further Reading

- Chapter 8 for the partition fork that makes these models bite.
- Chapter 4 for replicas as a scale lever.
- Chapter 12 for which stores make which promises easier.

Architectural trade-offs

Learning Objectives

- Say a complete trade-off: choose X because of Y; give up Z; mitigate with W.
- Pair the Part I forces: scale, availability, reliability, performance, consistency, cost, operability.
- Avoid false dichotomies and “best practice” with no loser.
- Use the sentence in the interview close.

Quote

> *If you cannot name what you gave up, you did not make a decision. You made a collage.*

Introduction

Part I taught you separate lenses. Real designs smash them together. This chapter is the architect’s sentence — the thing interviewers remember — and the end of Foundations. Building blocks in Part II are how you implement a choice you can already say out loud.

Core Concepts

The sentence: “I am choosing X because of constraint Y. The cost is Z. We mitigate with W.”

Examples you should be able to produce without a slide:

Choice	Because	Give up	Mitigate
Cache mappings	Tiny working set, read skew	Freshness of seconds	TTL plus invalidate on write
302 not 301	Need to change targets and count clicks	CDN offload of redirects	Cache the mapping, not the 301
Queue clicks	Protect redirect p99	Immediate exact counts	Approximate + lag metric
Read replica	Read QPS	Linearizable reads	Session read-your-writes on primary
Fail open on limiter Redis down	Homepage availability	Brief abuse window	Fail closed on payments
Single region first	Team of four, latency inside one geography	Disaster story	Document the 10× multi-region hour
Larger batches	Throughput of a pipeline	Per-item latency	Cap batch time, flush on size or timer
More app clones	Scalability under load	Does not fix single-user slowness	Profile the path first (Ch 4 vs Ch 7)

Those last two rows are **latency vs throughput** and **performance vs scalability**. Operability and cost remain first-class axes; the three pairs are the start of the table, not the end.

False dichotomies: SQL vs “NoSQL” as morality; microservices as scale; multi-region as availability. Trade-offs are about **this verb, these numbers, this team**.

Reversibility. Prefer the choice you can undo in a quarter. That is an enterprise architect’s bias and a valid interview sentence.

Architecture Diagram

Trade-offs live on the same path; they are labels on arrows, not new products.

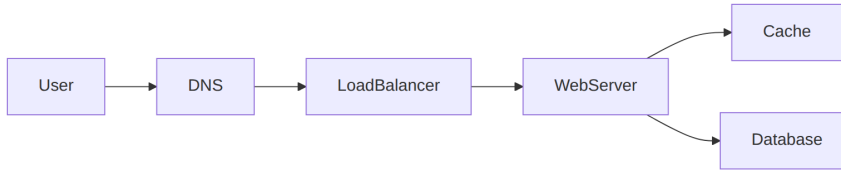


Figure 10.1 — Every extra arrow is a trade-off: cache vs freshness, extra hop vs p99, replica vs consistency. Annotate two of them. That is the chapter.

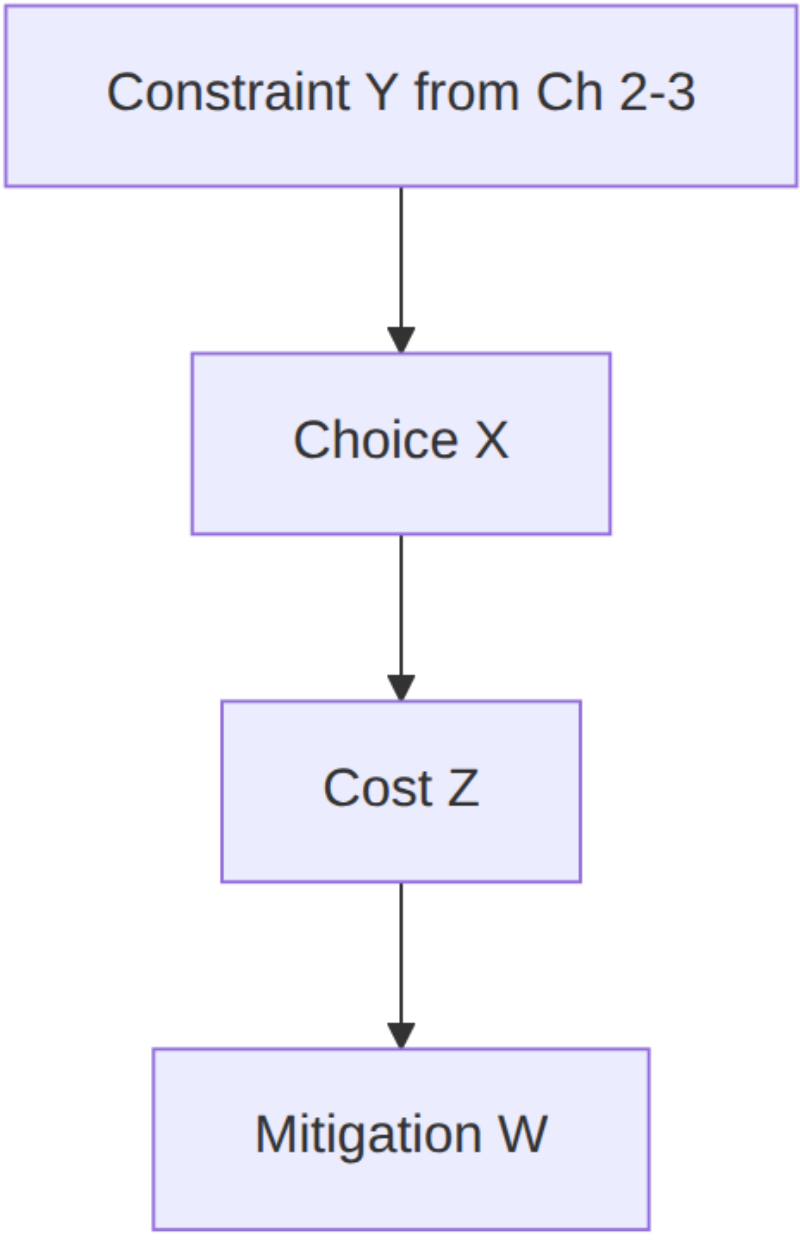


Figure 10.2 — The four-part sentence as a diagram. If W is "hope," the trade-off is unfinished.

Real-world Example

WhatsApp-like send path: you choose **store-and-forward in the sender's region first** because offline delivery is a must-have (Chapter 2) and connection RAM is the bottleneck (Chapter 3). You give up immediate global linearizability of the thread (Chapter 9). You mitigate with causal order per conversation and read-your-writes on the sending device. That is a complete trade-off. "We will use a globally distributed strongly consistent store" is not a trade-off; it is a wish that fights Chapter 7.

Enterprise Insight

In a company the trade-off table has extra columns: vendor lock-in, skill of the owning team, license cost, and control mapping. The "best" consistency model that nobody on the team can operate is a reliability incident (Chapter 6). Architects pick the design the organization can run at 3 a.m. Say that. Interviewers for senior roles are listening for operability as a first-class axis, not an apology.

Interviewer's Mind

They are waiting for the sentence. If you never give up anything, they will force you: "Your cache is down — now what?" Practice saying Z before they ask. The close (Chapter 18) should repeat one trade-off, not introduce a new one.

AI Perspective

"Add a model" is a trade-off: quality of ranking versus p99, cost, and privacy. Mitigation: async re-rank, fail open to unranked lists, keep plaintext off the vendor if E2E or residency forbids it. If you cannot name Z, the AI box is decoration.

Common Mistakes

- "Best of all worlds" diagrams.
- Trade-offs that are just product names.

- Mitigations that are the opposite choice in disguise.
- Changing the trade-off in the last minute.

Best Practices

- Write Y from requirements and numbers, not from fashion.
- One primary trade-off per deep dive.
- Operability and cost are allowed axes.
- Reversible first.

Summary

Foundations end when you can choose. The sentence — X because Y, cost Z, mitigate W — is the architect's unit of work. Part II gives you more X's. It does not replace the sentence.

Key Takeaways

- No loser, no decision.
- Map choices to Part I forces.
- Mitigate; do not deny Z.
- Senior interviews score operability as a trade-off axis.

Interview Questions

- Give the four-part sentence for adding a cache to redirects.
- What do you give up with read replicas, and how do you mitigate?
- Name a trade-off that is about the team, not the traffic.

Further Reading

- Chapters 2–9 as the source of Y and Z.
- Chapter 15, where this sentence sits in the interview loop.

Part II — Building Blocks

Caching

Learning Objectives

- Place a cache on a path only after a working-set argument.
- Choose aside versus read-through and say how you invalidate.
- Name aside, write-through, write-behind, and refresh-ahead as update styles.
- Name stampede and stale-read trade-offs.
- Keep user-visible writes honest when a cache sits in front.

Quote

> *Cache is not a database with optimism. It is a bet that yesterday's answer is still good enough for this request.*

Introduction

Caching is the most over-drawn box in interviews. Used well, it is the cheapest scale lever for read-heavy working sets. Used poorly, it is a second source of truth you cannot explain. This chapter is when the box earns its keep.

Core Concepts

Where it lives: client, CDN, application, datastore. Closer to the user is faster and harder to invalidate.

What you store: the object the hot path needs, not the event stream you will analyze later.

Aside vs read-through: aside keeps the app in control; read-through hides load in the cache layer. Both still need a TTL or an explicit invalidation story.

Stampede: many misses at once after expiry. Mitigate with jittered TTLs, locking, or serving stale while you refresh.

How the cache is updated (pick one and name the loser):

- **Aside:** app reads the store on miss, then fills the cache. Simple. Cache can be stale until TTL or delete-on-write.
- **Write-through:** write hits cache and store together. Reads are warmer; writes are slower.
- **Write-behind:** write hits cache, store catches up asynchronously. Fast writes; you can lose the last writes if the cache dies (reliability).
- **Refresh-ahead:** refresh before expiry. Smooths load; more background work.

Layers still matter: client, CDN, web tier, app, database. Closer to the user is faster and harder to invalidate.

Architecture Diagram

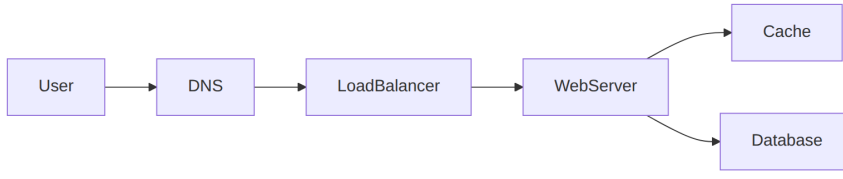


Figure 11.1 — The cache sits on the read path. Writes still land on the database; then you invalidate or overwrite the key. Do not draw arrows that imply the cache is durable.

Real-world Example

Redirect lookups for short links: tiny keys, tiny values, brutal read/write skew. Cache the mapping. Do not cache the click counter in the same way if you need approximate-but-durable analytics; send clicks async.

Enterprise Insight

Enterprises already have a CDN and often a shared Redis platform. Use them, but **do not share a cache cluster between unrelated domains** without key prefixes and memory budgets.

A noisy neighbor eviction is an availability incident. Data classification matters: putting session tokens and public catalog pages in the same store without TTL policy will fail a security review.

Interviewer's Mind

They want the sentence: "working set fits; TTL is N; stale is acceptable because…" They will poke invalidation. "Delete on write" plus TTL is enough for most 45-minute designs. Do not invent a research-grade coherence protocol unless they ask.

AI Perspective

Prompt caches and embedding caches are still caches: they trade freshness of context for latency and cost. A stale embedding after a document update is a correctness bug in search, not a minor miss. Separate "byte cache for the same URL" from "semantic cache for the same question" — the latter can return the wrong user's answer if keys are poorly scoped.

Common Mistakes

- Caching writes that must be right now.
- No TTL and no invalidation.
- One global key for a personalized page.
- Using the cache as the system of record.

Best Practices

- Justify with working set and ratio.
- Jitter TTLs.
- Key by the lookup the hot path actually does.
- Measure hit rate; in the interview, name that metric.

Summary

A cache is a freshness bet on a small, hot working set. Put it on the read path, invalidate on purpose, and never confuse it with durable storage.

Key Takeaways

- Working set first, Redis second.
- Invalidation is the design. Name the update style (aside, through, behind).
- Stampede is a load event you can plan for.
- Personalized data needs personalized keys.

Interview Questions

- When is a CDN enough and an application cache unnecessary?
- How do you avoid a stampede at expiry?
- What do you cache for a URL shortener, and what do you not?

Further Reading

- Chapter 3 for the numbers that justify the cache.
- Chapter 14 when a cache pool must survive add/remove without a miss storm.
- Chapter 16 for a worked redirect cache.

Data storage

Learning Objectives

- Choose storage by access pattern and consistency needs, not by fashion.
- Separate the system of record from projections (search, cache, warehouse).
- Explain primary keys, secondary lookups, and why blobs do not belong in the OLTP row.
- Speak replication and partitioning without hiding behind a vendor name.

Quote

> *The database is not "where data goes." It is the component whose consistency model you are actually shipping.*

Introduction

Storage interviews go wrong when the candidate lists five databases to look experienced. You need one system of record per entity, and optional projections that can be rebuilt. This chapter is that discipline.

Core Concepts

Access pattern first. Lookup by primary key, range scan, full-text, time series, and blob download are different jobs. One engine rarely wins all of them.

System of record vs projection. Search indexes and warehouses are derived. If they disagree with the record, the record wins unless you explicitly designed a different rule.

Keys. The unit of consistency is usually a row or a document with a key. Secondary indexes are not free. Hot partitions follow bad keys.

Blobs. Large objects go to object storage; the database keeps the pointer and the metadata.

Scale levers on a relational record (when the numbers demand them, Chapter 4):

- **Replication:** extra copies for read scale or failover. Consistency cost is Chapter 9.
- **Federation:** split by function (users DB vs orders DB). Joins across them become your problem.
- **Sharding:** split by a key inside one function. Hot keys remain.
- **Denormalization:** copy fields to avoid joins. Writes must update several places or you accept lag.

SQL versus a specialized store is still access-pattern first, not a fashion choice.

Architecture Diagram

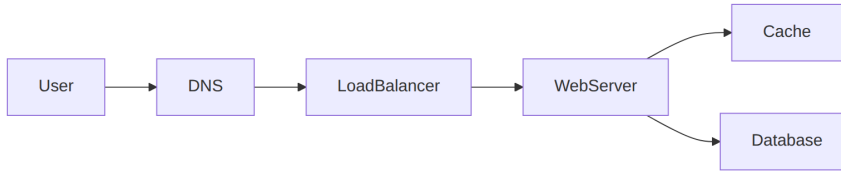


Figure 12.1 — Database on the write path; cache in front for hot reads. If you add search or object storage, they branch off this picture as extra stores with a stated role, not as second sources of truth.

Real-world Example

Chat messages: the conversation's recent messages are the working set (fast store keyed by conversation ID). Media is object storage plus a metadata row. Search-across-all-history is a projection filled asynchronously. One "chat database" that tries to do all three will satisfy none.

Enterprise Insight

You will inherit a licensed warehouse, a mandated encryption module, and a backup SLA measured in hours. Design for those. “We’ll switch off Postgres” is not an enterprise move in a first interview unless they asked you to greenfield. Do mention: backup/restore drills, encryption keys, and who owns schema change. Multi-region active-active is a product and a legal decision, not a checkbox on a managed database.

Interviewer’s Mind

They like “Postgres is enough” when the numbers say so. They like “this lookup is by short code, so a KV store is justified.” They dislike a graph database appearing because the word “relationship” was used once. Ask what queries must be fast before you pick the engine.

AI Perspective

Vector indexes are projections of embeddings. The document bytes remain in object storage or a document store. Rebuild embeddings when the source changes. Do not put the only copy of a contract inside a vector database. Training data residency is a storage NFR: same questions as any other PII store.

Common Mistakes

- One store for transactional rows, blobs, and analytics.
- Secondary indexes on write-heavy keys without a plan.
- “We’ll use NoSQL for scale” with no access pattern.
- Forgetting restore, not just backup.

Best Practices

- Name the system of record.
- Put blobs in object storage.
- Match engine to query.
- State replica lag if you read from replicas.

Summary

Storage follows access pattern and consistency. Keep one record, derive the rest, and keep large bytes out of the transactional row.

Key Takeaways

- Pattern first, product second.
- Replication, federation, sharding, and denormalization are levers with costs, not decorations.
- Projections can be rebuilt; records cannot.
- Keys drive both correctness and hot spots.
- Object storage for blobs is a default, not a flourish.

Interview Questions

- When is a relational primary the right call at 10k QPS?
- How do you delete a user across record and projections?
- Why might you refuse a graph database in the first pass?

Further Reading

- Chapter 4 on partition keys; Chapter 14 when shard owners change with the fleet.
- Chapter 13 on filling projections asynchronously.
- Chapter 9 on which consistency model that store actually offers.

Messaging and asynchronous work

Learning Objectives

- Move work off the hot path when the user should not wait.
- Choose queue versus log in first-principles language.
- Design consumers for at-least-once delivery and idempotent writes.
- Apply backpressure instead of unbounded retries.

Quote

> *If the user is not waiting for it, it is a message. If they are waiting for it, it is still a request — even if you put a broker in the middle.*

Introduction

Asynchronous processing is how you protect the latency budget you promised in Chapter 2. Click accounting, thumbnails, emails, search indexing: none of these should sit on the redirect or the checkout round-trip. This chapter is the extra arrow on the diagram — the dashed one.

Core Concepts

Queue. Competing consumers pull work. Good for tasks. Ordering across the whole system is not the point.

Log. Consumers keep an offset. Good when multiple independent projections must see the same events.

Delivery. At-least-once is the default you should assume. Exactly-once is a marketing phrase unless you designed idempotency and a transactional outbox.

Backpressure. If consumers are slow, the buffer grows. Decide in the interview: slow the producer, drop with a metric, or shed load. An unbounded queue is not reliability; it is a delayed outage. Task queues (run this job) and pub/sub logs (many independent readers) both need a lag SLO.

Silence is how you discover the incident in billing.

Architecture Diagram

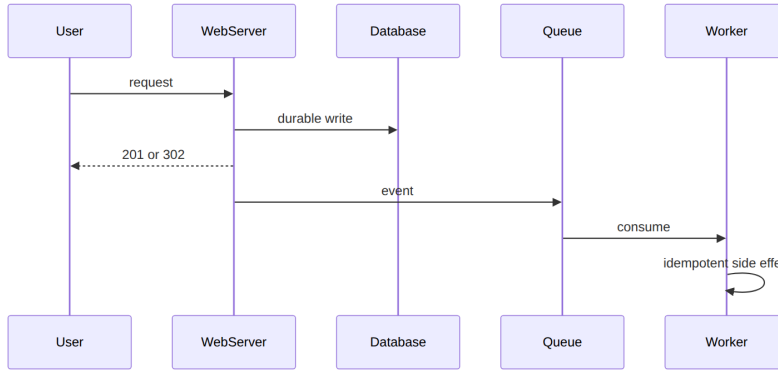


Figure 13.1 — User-visible write lands first. The event is after success (or in the same transaction via an outbox). The worker must tolerate duplicates.

Real-world Example

Short-link clicks: the user needs a 302. The click is an event. If the analytics store is down, redirects still work. You will replay the queue. Duplicate clicks may over-count unless you key by (code, user, time bucket) or accept approximate counts — say which.

Enterprise Insight

Enterprises have a mandated broker and a fear of “yet another Kafka.” Use the platform bus if it exists; isolate with topics and ownership, not with a shadow cluster you cannot operate. Poison messages need an operator path, not an infinite retry that pages the wrong team. Audit often requires that the event be as trusted as the HTTP log; do not emit events you cannot reconstruct.

Interviewer's Mind

They want the dashed line and the words "at-least-once" and "idempotent." They will ask what happens if the worker dies after sending the email but before acking. Have that sentence ready. Do not introduce a broker for a 50 QPS CRUD API with no fan-out.

AI Perspective

Inference batches and embedding jobs are async by default. Queue GPU work; do not hold an HTTP request while a 70B model thinks unless the product is that wait. For user-facing chat with a model, you may stream tokens on a request path instead of a broker — choose on latency, not on fashion.

Common Mistakes

- Event before durable write (lost intent).
- Assuming ordering across partitions.
- Unbounded queues as "reliability."
- Dual writes to database and broker without an outbox.

Best Practices

- Write record, then event.
- Idempotency keys on side effects.
- Dead-letter with an owner.
- Metric: lag, not just throughput.

Summary

Async work is how you keep promises about latency. Assume duplicates, make side effects safe to repeat, and watch lag.

Key Takeaways

- User wait versus not is the split.
- At-least-once plus idempotency is the default pair.
- Logs and queues solve different fan-out problems.

- Lag is an availability metric for projections.

Interview Questions

- Why is “exactly once” the wrong first sentence?
- When would you refuse to add a message broker?
- How does an outbox prevent lost events?

Further Reading

- Chapter 5 for timeouts versus queued work.
- Chapter 16 for click events off the redirect path.

Consistent hashing

Learning Objectives

- Explain why `hash(key) % N` causes a miss storm when N changes.
- Place keys and servers on a hash ring and look up clockwise.
- Show that add/remove of a node remaps only nearby keys.
- Use virtual nodes to even out partition sizes.

Quote

> *A partition function that depends on the size of the fleet is a partition function that will lie to you on the worst day of the year.*

Introduction

Horizontal scale (Chapter 4) needs a rule: this key lives on that box. The naive rule is modulo the current server count. It is even when the fleet is frozen. It is a disaster when a box dies, because **almost every key** hashes to a new index. Caches go cold. Databases reshuffle. Consistent hashing is the interview name for a lookup that keeps most keys where they were.

This chapter is that rule, in first principles: a ring, clockwise lookup, virtual nodes. It is how you shard caches, request routers, and some data stores without pretending N is eternal.

Core Concepts

The rehashing problem

With N servers, `server = hash(key) % N` is uniform if the hash is. Fetch is cheap. Add or remove one server and N changes, so **the same key** maps to a different remainder. Not only the keys that

lived on the dead box move — most keys move. Clients then ask the wrong cache. That is a coordinated miss storm, not a graceful failover.

Hash space as a ring

Pick a hash with a large output space (think 0 .. 2^160-1 if you name SHA-1 in the room; the exact function matters less than **no modulo by N**). Treat the space as a circle by joining min and max. Map **servers** (by name or IP) onto the circle with the same hash. Map **keys** onto the same circle. To find a key's owner, walk **clockwise** from the key until you hit a server.

Add and remove

- **Add a server:** it lands between two existing servers. Only keys in the arc that now hits the newcomer move to it. Everyone else stays.
- **Remove a server:** its keys walk clockwise to the next live server. Other arcs are untouched.

On average you remap about k/n keys when one of n slots changes — not k. That is the property you are buying.

Uneven arcs

Real hashes do not space servers perfectly. One dead neighbor can leave a survivor with an arc twice as wide as its peers. That box then owns too much load.

Virtual nodes

Give each physical server **many** positions on the ring (virtual nodes), all labeled with the same owner. Arcs become small and mixed. When a machine dies, its virtual nodes' neighbors (often several different machines) absorb the keys. When you add capacity, you sprinkle new virtual nodes instead of one lucky gap. The virtual count is a tuning knob (tens to hundreds per box in production; three is enough to *draw*).

Architecture Diagram

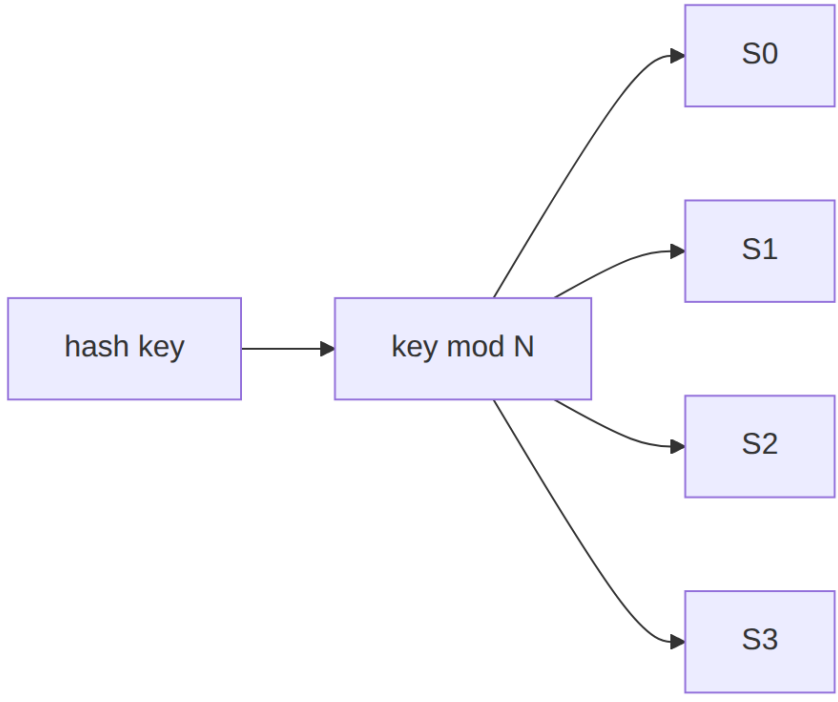


Figure 14.1 — Modulo placement. Fine while N is fixed.

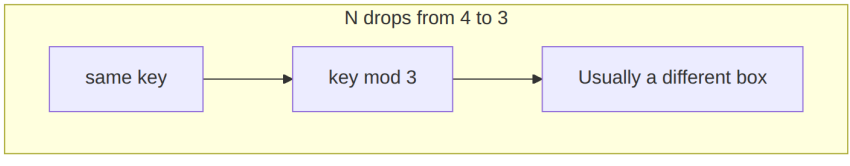


Figure 14.2 — The rehashing problem: most keys change owner, so caches miss in a storm.

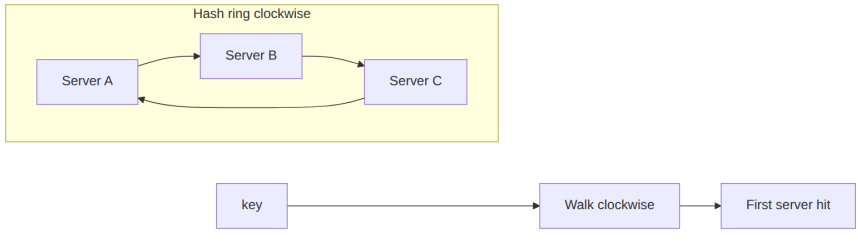


Figure 14.3 — Lookup: no modulo by fleet size. Walk the ring until a server.

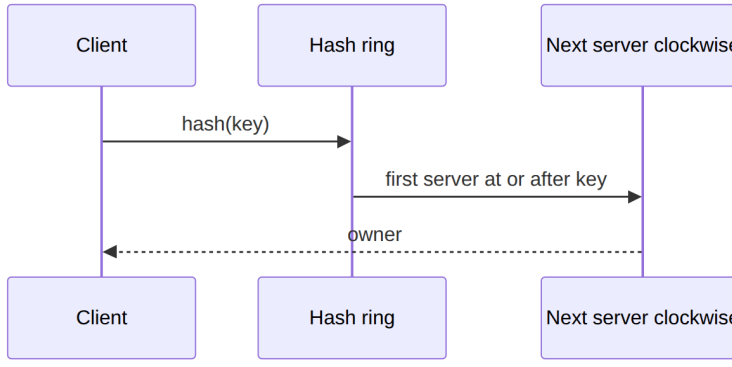


Figure 14.4 — Add a node: only the preceding arc moves. Remove a node: only its arc moves to the next clockwise owner.

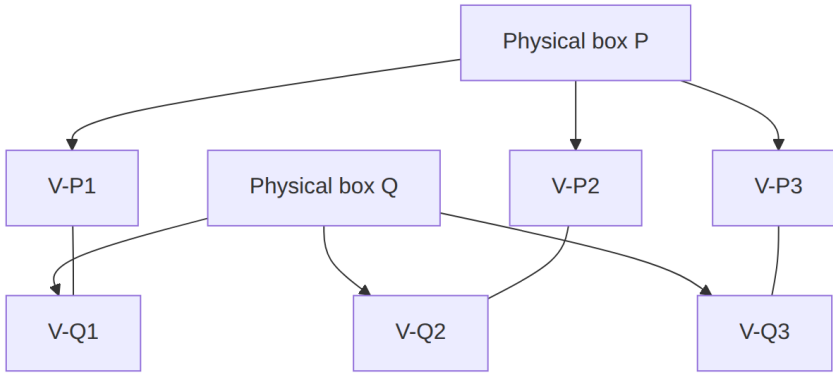


Figure 14.5 — Virtual nodes: one machine, many ring points, smaller unevenness when someone leaves.

Real-world Example

A session cache of eight nodes uses `hash(user) % 8`. One node OOMs at peak. After it is pulled, N=7 and almost every session misses. Login looks like an outage even though seven caches are healthy. Consistent hashing plus virtual nodes would have moved only the dead node's share, spread across several survivors. The product lesson: **cache sharding that depends on N is an availability bug**.

Enterprise Insight

Platform Redis and Kafka-style partitions already encode this idea (or a relative). Your job in review is to ask: what happens when we add a cache pod at 14:00 on a weekday? If the answer is “we flush everything,” you do not have consistent hashing — you have a maintenance window. Also ask who owns the hash library. Two teams with two rings and the same keys will split brains. Sticky sessions at the load balancer are a poor substitute: they fight scale (Chapter 4) and still reshuffle on deploy.

Interviewer’s Mind

They want the modulo storm named **before** the ring. Weak: “we will use consistent hashing” as a product sticker. Strong: draw four boxes, kill one, show which keys move, then add virtual nodes when they ask about imbalance. If they ask about databases, say this is a **placement** function; it does not replace replication or a primary.

AI Perspective

Embedding indexes and GPU pools also need a placement function. Modulo by replica count will reshuffle vectors on every scale event — expensive re-embeds. Consistent hashing (or a managed shard map) keeps most embeddings put. Do not put the model weights themselves on a key ring unless you enjoy loading 70B parameters because one pod died.

Common Mistakes

- Modulo by live count in the request path.
- One ring position per fat machine, then wondering why load is skewed.
- Treating the ring as a consistency model.
- Forgetting to rebuild the in-memory ring when membership changes (now you *do* miss).

Best Practices

- Hash servers and keys with the same function; never `% N` for ownership.
- Clockwise (or consistently counterclockwise) lookup, documented.
- Virtual nodes from day one if boxes are few or unequal.
- Membership change is a control-plane event; measure miss rate around it.

Summary

`hash % N` is fair until N moves, then it is a miss storm. A hash ring plus clockwise lookup remaps only a neighbor's keys. Virtual nodes keep arcs honest. Use it when you shard caches or request owners across a changing fleet.

Key Takeaways

- Modulo by fleet size remaps almost everything on add/remove.
- Ring lookup remaps about 1/n of keys.
- Uneven arcs are the remaining problem; virtual nodes shrink them.
- Placement is not durability.

Interview Questions

- Why do most cache keys miss after one node of a `% N` pool dies?
- Walk a key clockwise onto a four-node ring, then add a fifth. Which keys move?
- What does a virtual node actually store?
- When would you refuse consistent hashing and keep a static map?

Further Reading

- Chapter 4, the scale lever this placement function serves.

- Chapter 11, why a miss storm is an availability event.
- Chapter 12, sharding keys when the store — not only the cache — must split.

Part III — The Interview

The interview method

Learning Objectives

- Run a seven-move loop in 45 minutes without losing the plot.
- Produce four artifacts: scoped problem, justifying numbers, one diagram, one trade-off.
- Use Mermaid for time-ordered behavior and Draw.io for layered architecture.
- Recover when the interviewer steers.

Quote

> *If you get lost, return to the last box you actually completed.*

Introduction

A system design interview is a short, public architecture review. You do not need every pattern in the industry. You need a **loop** you can run when the problem is vague and the clock is not. This chapter is the spine of the book. Parts I and II were fuel. This is the engine.

Core Concepts

Seven moves:

1. **Restate and bound** — what we are building, who it is for, what we are not building.
2. **Requirements** — functional first, then the NFRs that change the design.
3. **Back-of-the-envelope** — QPS, storage, bandwidth, so the boxes have sizes.
4. **Contracts** — APIs and the core data model.
5. **High-level design** — clients, edge, app, data, async. One picture.

6. **Deep dives** — two or three places the design can fail or get expensive.
7. **Scale the story** — what breaks at 10×, and what you would do next week versus next year.

Time budget (default, not law):

Minutes	Move	Output on the board
0–5	Restate and bound	Problem sentence + out of scope
5–12	Requirements	5–8 bullets, 2–3 NFRs highlighted
12–18	Estimates	QPS, size of hot data, 1–2 “so we need…”
18–22	Contracts	4–6 endpoints or events, 2–3 entities
22–32	High-level design	One Draw.io-style diagram
32–42	Deep dives	Bottleneck, consistency, failure
42–45	Close	Risks, metrics, what more time buys

A common four-step flow (scope and assumptions → high-level sketch → core components → scale and bottlenecks) is the same conversation compressed. Our seven moves just make estimates and the close first-class. Do not switch templates mid-interview.

Architecture Diagram

Figure 15.1 — The interview loop.

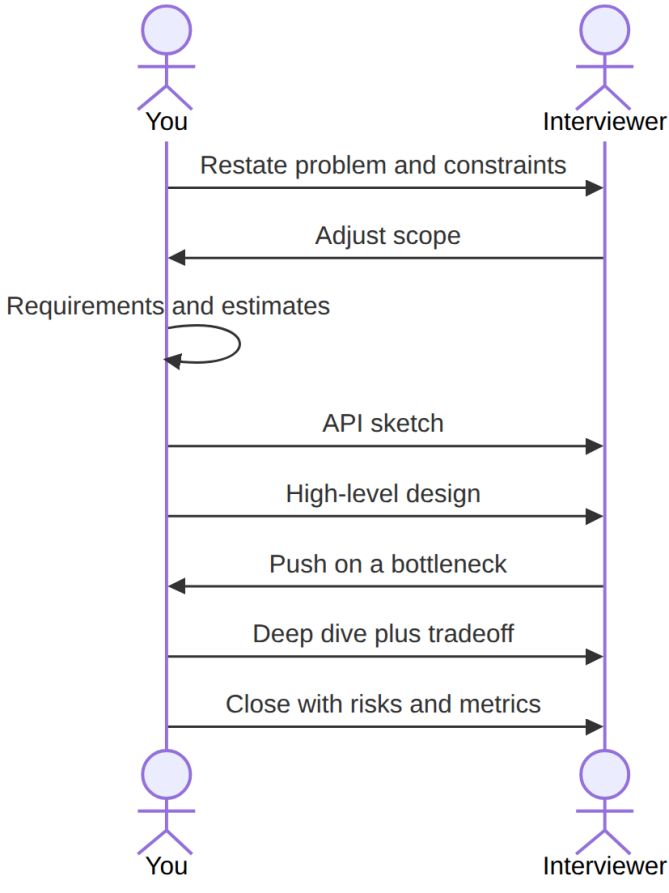
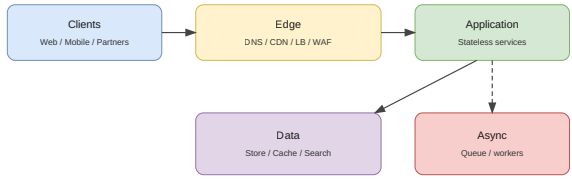


Figure 15.2 — You lead; they steer.

When the design has more than three boxes, draw layers. Copy the Draw.io template:



Playbook high-level template

Figure 15.3 — Default layers: clients, edge, application, data, async. Rename boxes; do not skip a layer without a sentence.

Real-world Example

“Design a URL shortener” at speed (full version in Chapter 16): restate create+redirect; park custom domains; NFRs are redirect latency and uniqueness; estimates show read-heavy tiny mappings versus bulky click logs — split the paths; deep dive code generation and cache. If the pretty diagram is incomplete, finish the **redirect sequence**.

Enterprise Insight

This loop is the same skeleton as an architecture review with extra time removed. In enterprise rooms you add: who operates it, which platform services are mandatory, which control is non-negotiable. In the interview, mention those if they would change a box. Do not perform governance theater.

Interviewer’s Mind

They are scoring whether you can be steered without collapsing. Every interruption is a gift. If they ask about cache stampede, that *is* the deep dive; do not finish your unused talking points first. Weak close: new database in the last ninety seconds. Strong close: two risks, two metrics.

AI Perspective

Do not use a model in the room as a substitute for the loop. Afterward, ask a model to attack your diagram: missing timeout, missing key, missing degraded mode. In a product that is itself AI, the loop does not change — the model is a dependency with a budget.

Common Mistakes

- Inventory dump instead of a loop.
- Diagram with no read/write path.
- Skipping estimates, then inventing shards.
- Ignoring the interviewer to “finish the design.”

Best Practices

- Name skipped steps so they know you did not forget.
- Four artifacts or you are not done: scope, numbers, picture, trade-off.
- Draw while talking.
- Park extra features visibly.

Summary

Run the loop. Leave four artifacts. Let the interviewer choose the deep dive. Close without a new invention.

Key Takeaways

- Seven moves, one clock.
- Picture plus trade-off beats a catalog.
- Steering is part of the method.
- Stop when the artifacts exist.

Interview Questions

- What do you drop first if you have fifteen minutes left?
- How do you show you skipped APIs on purpose?
- Which four artifacts should remain if the board is photographed?

Further Reading

- Appendix A.
- Chapters 2–10 as the fuel for the loop; Chapter 14 when membership of a cache or shard pool changes.
- Chapters 16 and 17 as the first two fully worked problems.

Problem: URL shortener

Learning Objectives

- Apply the interview loop to create-and-redirect.
- Split the redirect hot path from click analytics.
- Size a short code (alphabet and length) from capacity, then pick hash-plus-retry or counter-plus-encode.
- Defend cache and 302 versus 301 as trade-offs, not trivia.

Quote

> *The user is waiting on the redirect. Everything else is a dashed line.*

Introduction

This is the first fully worked problem. The product sounds small. It is a clean test of requirements, tiny-but-hot data, and the temptation to overdraw. You will run Chapter 15 on a specific verb: turn a long URL into a short one, then send people back.

Core Concepts

Functional: given a long URL, return a short one; given the short one, send the user to the long one; optional click counts.

Out of scope unless pulled in: custom domains, QR, campaign suite, SSO.

NFRs that change the design: low latency on GET, uniqueness of codes, durability of the mapping, read-heavy ratio.

301 versus 302

301 says the move is permanent. Browsers and CDNs may never ask you again — good for your CPU, bad if you need to change the target or count clicks. **302** says temporary: every click still visits you. Default to **302** unless they explicitly want edge offload of redirects.

How long is the code?

Use a 62-character alphabet: `0-9`, `a-z`, `A-Z`. Length n gives 62^n possible codes. Pick the smallest n that clears your Chapter 3 estimate with room for collisions and retired codes. For example, n = 7 is about 3.5×10^12 strings — far above a few hundred billion mappings. Say the estimate **before** you pick seven. Do not recite a magic number.

Hash then resolve collisions

A cryptographic hash of the long URL is longer than seven characters. Taking a prefix is fine if you **treat collisions as a data problem**: insert with a unique index; on conflict, mix in extra entropy (a counter or a salt) and try again. A bloom filter can skip some disk checks; it is optional polish. The uniqueness constraint is not optional.

Counter then encode

A monotonic ID encoded in base-62 also works. You need a uniqueness authority (SQL sequence, Snowflake-style ID from Chapter 12's key story). Collisions vanish; IDs leak volume.

Architecture Diagram

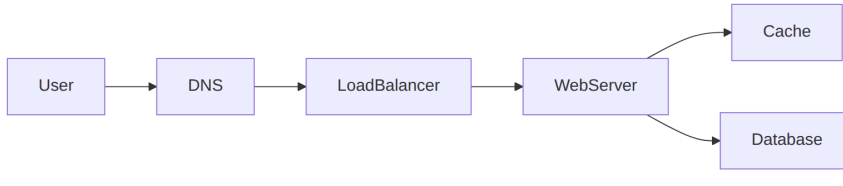


Figure 16.1 — Redirect read path. The web server here is the redirect service.

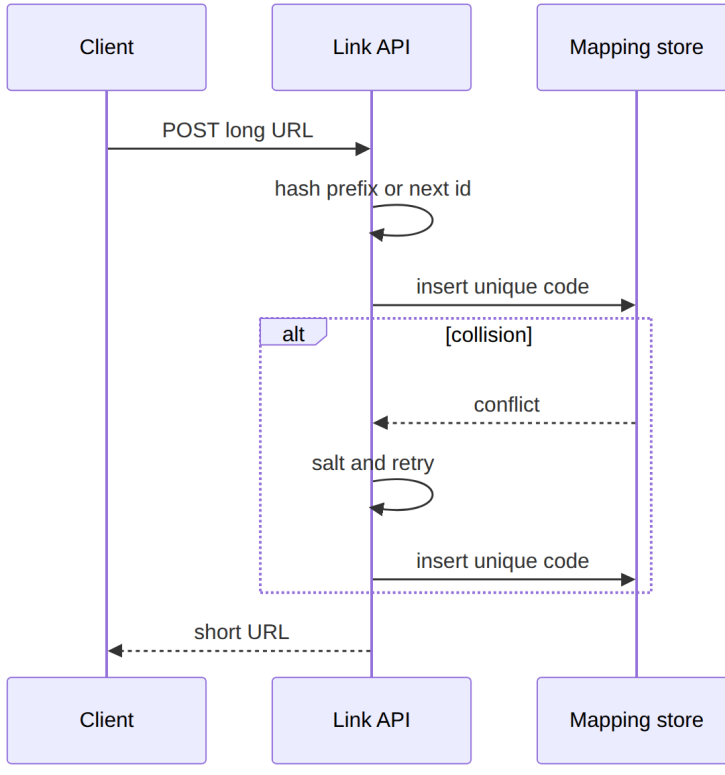


Figure 16.2 — Shortening. Uniqueness lives in the store, not in hope.

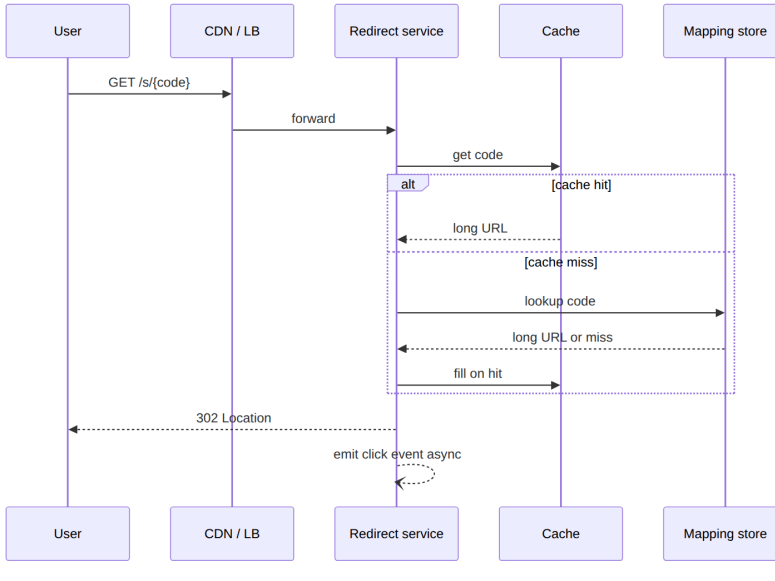
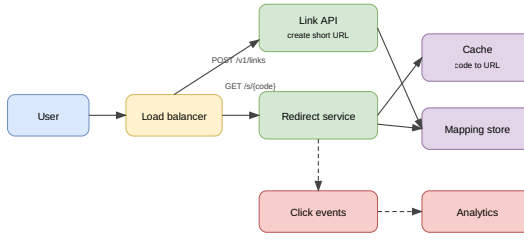


Figure 16.3 — Happy-path redirect. Unknown codes 404. Analytics must not sit on the latency budget.

Polished layered view:



URL shortener high-level

Figure 16.4 — Writes to the mapping service; reads hit cache then store; clicks are asynchronous.

Real-world Example

A marketing team wants to change the destination of a printed code after print. That single requirement kills permanent 301 caching at the edge and makes the mapping store the source of truth. Click counts can be approximate; destination changes cannot.

Enterprise Insight

Enterprises will ask about abuse (open redirector), retention of click logs (PII if you store IPs), and whether the short domain is a brand asset on the corporate DNS. Put rate limits at the edge (Chapter 17). Put logs in a store with a deletion story. Do not run a public shortener without an acceptable-use path; interviewers like that you mentioned it. If you shard mappings, place them with consistent hashing (Chapter 14) or a static range — do not `% N` the code.

Interviewer's Mind

They expect the cache, the uniqueness story, and the async click. They may push on "what if we have 100× clicks in one country" (edge cache of 302 responses is still risky if destinations change) or "how do you guarantee no collision." Weak: only hashing without retry. Strong: "I will retry on unique-index violation." They may ask 301 vs 302 to see if you think about analytics versus load.

AI Perspective

AI does not belong on this hot path. Where it might appear later: abuse classification of created URLs, run async, with a fail-open redirect if the classifier is down so you do not take the product down to catch phishing late.

Common Mistakes

- Analytics on the redirect round-trip.
- 301 by default.
- No uniqueness constraint in the store.
- Encoding secrets in the short code and calling it security.
- Picking seven characters with no 62^n story.

Best Practices

- Unique index on code; retry on conflict.
- Cache mappings, queue clicks.
- 302 until proven otherwise.
- Rate-limit create.
- Size n from capacity, not from folklore.

Summary

A URL shortener is a tiny mapping with a huge read skew. Size the alphabet, enforce uniqueness, protect the redirect path, and keep scorekeeping off the user wait.

Key Takeaways

- Mapping is small; click logs are not.
- Uniqueness is a store constraint, not a hope.
- 302 preserves control; 301 donates it to the browser.
- Dashed line for analytics.

Interview Questions

- Hash versus counter: what do you give up with each?
- Why might a CDN-cached 301 be a product bug?
- How do you handle a collision on insert?
- Why 62^7 and not 62^5 for your estimate?

Further Reading

- Chapter 11 (cache) and Chapter 13 (click events).
- Chapter 14 if mappings must split across boxes.
- Chapter 17 if they add “please stop bots creating links.”

Problem: Rate limiter

Learning Objectives

- Place rate limiting at client, server, or gateway and say why the client is not enough.
- Compare token bucket, leaky bucket, fixed window, and sliding windows in trade-off language.
- Store counters in memory with a key and a TTL, not on the disk path.
- Return 429 plus remaining/limit/retry headers, and pick fail-open versus fail-closed.

Quote

> *A rate limiter is how you spend availability on purpose so someone else cannot spend it for you.*

Introduction

Rate limiting is a small design with large production consequences. It protects error budgets, noisy neighbors, and paid downstreams. Apply the loop: who is limited (IP, user, API key), on which verb, how precise, where the counter lives, and what the caller sees when they are over.

Core Concepts

Why bother

- Starve less: one client cannot eat the error budget (Chapter 5).
- Cost: third-party APIs billed per call.
- Load: bots and retries should die at the edge, not in the app.

Where it lives

- **Client:** easily forged; do not trust it as enforcement.
- **Server library:** full control of the algorithm; every language and instance must share state or you under-count.
- **Gateway / middleware:** one choke point, 429 before the app. Prefer this when an API gateway already does TLS and auth.

Algorithms (pick one and name the loser)

Token bucket. Tokens refill at a steady rate up to a cap. A request spends a token. Bursts are allowed until the bucket is empty. Good default for APIs that may spike briefly.

Leaky bucket. Queue requests and drain at a fixed rate. Smooth outflow; a burst fills the queue with old work and delays the new.

Fixed window counter. Count in wall-clock bins (per second, per minute). Cheap. Two bursts on either side of a boundary can admit almost 2× the budget.

Sliding window log. Store timestamps; drop those outside the last T; admit if count < limit. Accurate, memory-heavy (even rejects may leave a stamp).

Sliding window counter. Blend this bin and the previous bin by overlap. Approximate, memory-light, good enough for most abuse control.

Distributed counters

Do not put the counter on a disk database. Use an in-memory store with increment and expire. Key by the limit identity (API key, IP, user). Across app boxes, **approximate** is acceptable for abuse; a linearizable global count is an outage waiting to happen (Chapter 8). Shard the counter key, not the world.

When they exceed

HTTP **429**. Headers that teach the client: remaining, limit, retry-after. Optionally enqueue (orders) instead of drop — that is a product choice, not a default. If the counter store is down: **fail open** on a brochure page, **fail closed** on payments and GPU (Chapter 7).

Architecture Diagram

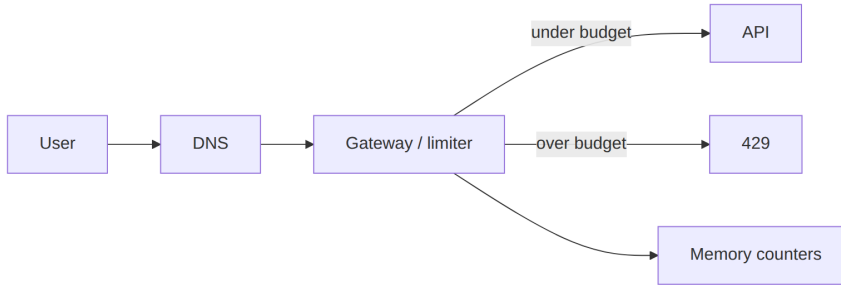


Figure 17.1 — Limiter in middleware. Counters are fast state, not the system of record.

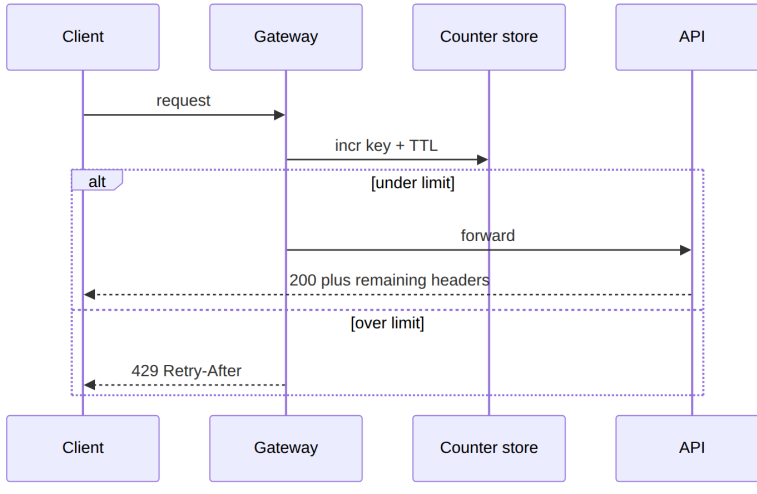


Figure 17.2 — Happy path increments; excess never reaches the app.

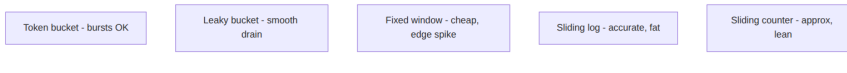


Figure 17.3 — Algorithm menu. State the burst behavior you want before you name one.

Real-world Example

The create-short-link API is expensive because of abuse, not because of CPU. Limit by API key at 10/s with a burst of 20 (token bucket). Redirects stay unlimited except for obvious bot nets, handled at the CDN. Different verbs, different budgets. Rules themselves live in config, cached in the gateway, not hardcoded in every service.

Enterprise Insight

Gateways already have rate-limit plugins. Prefer them to a custom service. Enterprises also need *quotas* (monthly) which are billing, not edge milliseconds — those live in a slower store. Distinguish burst control from quota. Legal may require that you do not lock out a hospital's NAT; have a bypass path for blessed keys. Monitor false drops: rules that are too tight are a self-inflicted outage.

Interviewer's Mind

They want a 429, a `Retry-After`, and honesty about approximation across instances. They may walk algorithms until you contrast burst versus smooth. Weak: "Redis INCR" with no key and no TTL. Strong: key, TTL, algorithm, fail policy. They may ask client versus server; say the client is a courtesy.

AI Perspective

Model endpoints need tighter, cost-aware limits (tokens per minute, not just requests). Separate the limiter for GPU APIs from the REST CRUD limiter. Fail closed on inference if cost is the risk; fail open on read-only catalog if availability is the risk.

Common Mistakes

- One global counter for the whole platform.
- Fail closed on the homepage because Redis blinked.
- Precision that requires a transaction per request.
- No distinction between user and IP.
- Fixed windows without mentioning the boundary burst.

Best Practices

- Name the key, the budget, the algorithm, and fail-open/closed.
- 429 plus remaining / limit / retry-after.
- Different limits per verb.
- Watch drop rate; loosen or switch algorithm if you are killing good traffic.

Summary

Rate limiting is a budget with a key, a window, a place on the path, and an algorithm whose burst behavior you can defend. Approximate distributed counts are fine for abuse. Errors should teach the client when to come back.

Key Takeaways

- Client limits are not enforcement.
- Token bucket for bursts; leaky for smooth; watch fixed-window edges.
- Counters in memory with TTL.
- 429 is a designed response, not an accident.

Interview Questions

- Token bucket versus leaky bucket: which burst behavior do you want?
- Why can a fixed one-minute window admit almost twice the quota?

- When must the limiter not share a cache with the application?
- Fail open or fail closed for payments versus marketing pages?

Further Reading

- Chapter 5 (error budgets and series components: a limiter in series can cost nines).
- Chapter 7 (do not spend GET latency to police create throughput).
- Chapter 16 (protect create, not redirect, by default).

Closing the interview

Learning Objectives

- Close in two minutes without a new invention.
- Leave risks, metrics, and a next-hour plan.
- Handle extra time with a planned deep dive, not a new database.
- Turn the session into a photograph the interviewer can replay.

Quote

> *Do not introduce a new database in the last ninety seconds.*

Introduction

Closings are where strong designs become memorable and weak ones leak. You already have the loop. This chapter is the last move: stop designing, start summarizing.

Core Concepts

The two-minute close: restated design in one breath, two risks, two metrics, what another hour would buy.

Risks are specific: hot key on `sku_id`, replica lag on read-your-writes, queue lag hiding stale search.

Metrics are user-visible plus system: p99 redirect, error rate, cache hit ratio, consumer lag.

Extra time. If they offer five minutes, deepen one existing box. Do not add a graph database.

Architecture Diagram

Figure 12.1 — The close is G plus a verbal photograph of E and F. You are not starting a new loop.

Real-world Example

You designed the shortener. Close: “Create and redirect, 302, unique index, cache mappings, async clicks. Risks: cache stampede on a popular code, collision retries under a naive hasher. Metrics: p99 redirect, unique-violation rate, click-queue lag. With another hour I would design custom domains and abuse classification, not a new store.”

Enterprise Insight

In a real steering committee you close with decision, owner, and review date. In an interview, the analog is: the trade-off you chose, the metric that would prove you wrong, and the reversible next step. Executives remember the last two minutes more than your middle boxes.

Interviewer’s Mind

They are writing feedback now. Help them: repeat the NFR that drove the design. If they are silent, ask “where would you like to go deeper?” That is confidence, not weakness. Do not apologize for the whole design.

AI Perspective

If you mentioned a model, close with its fail mode: “classifier is async fail-open.” If you did not need AI, do not bolt it on in the close to sound modern.

Common Mistakes

- New components in the close.
- Repeating the whole design at full length.
- No metrics.
- Talking until they cut you off.

Best Practices

- Timer in your head at minute 42.
- Two risks, two metrics.
- Point at the board; do not orate facing the interviewer only.
- Stop talking when the four sentences are done.

Summary

The close is a photograph: what you built, what can hurt it, how you would see that, and what time would buy. Repeat **one** trade-off from Chapter 10 (latency vs throughput, or consistency vs availability). Then silence.

Key Takeaways

- Minute 42 is a first-class design step.
- Risks and metrics outlive boxes.
- Extra time deepens, it does not expand.
- No new stores at the end.

Interview Questions

- What two metrics prove a URL shortener is healthy?
- How do you use leftover time without scope creep?
- What does a bad close sound like?

Further Reading

- Chapter 15, the loop this close finishes.
- Appendix A, last line.
- After each mock: write your actual closing sentences from memory and edit them.

Glossary

Keep definitions interview-short. Add terms as chapters land.

Availability. The share of time a service can do useful work inside its SLO.

Hot path. The request the user is waiting on.

Idempotent. Doing the same write more than once leaves the same stored result.

NFR. Non-functional requirement: latency, availability, cost, compliance, and the rest.

SLO. Service level objective: a target you can measure, not a slogan.

Working set. The data that must be fast, not the data that merely exists.

Latency. Time to finish one action.

Throughput. How many of those actions finish per unit time.

Nines. Availability spoken as 99.9% (three nines), 99.99% (four nines), and so on; convert to downtime before you promise them.

CAP (operational). When replicas cannot agree in time, do you refuse the verb or proceed and repair.

Eventual consistency. If writes stop, copies converge; until then a read may be stale.

Read-your-writes. The client that wrote a value can read that value back.

Reliability. Correct work over time, not merely “the process is up.”

Consistent hashing. Place keys and servers on a hash ring and walk clockwise (or always the same way) so adding or removing a node remaps only nearby keys, not `hash % N` of the whole set.

Virtual node. Extra ring positions owned by one physical server, used to even out arc sizes when membership changes.

Token bucket. Rate-limit algorithm: tokens refill up to a cap; a request spends a token; bursts are allowed until empty.

429. HTTP status for “too many requests”; pair it with remaining, limit, and retry-after headers.

Appendix A — Board cheat sheet

Use this when the clock is loud.

1. Restate the problem in one sentence. Name what is out of scope.
2. Functional requirements, then two NFRs that would change the diagram.
3. Rough QPS, payload, stored bytes, read/write ratio. Convert with powers of two. Check hops against latency orders of magnitude (Chapter 3).
4. Four to six APIs or events. Two or three entities and their keys.
5. One picture: clients, edge, app, data, async.
6. Deep dive the bottleneck the numbers pointed to. Name a trade-off pair: performance vs scale, latency vs throughput, or availability vs consistency. If you shard a cache or mapping store, say whether ownership is `% N` or a hash ring (Chapter 14).
7. Close with two risks, two metrics, and what another hour would buy. If create is abuse-prone, name the 429 path (Chapter 17) without putting it on the user-facing GET.

References

Use this list for *concepts only*. Do not paste wording, figures, or question banks from these sources into chapters.

- Your own production postmortems and architecture decision records.
- Public cloud well-architected material (reliability, cost, performance pillars) as a checklist of concerns, not as text.
- Foundational papers you actually read (for example on replication, consensus, or stream processing) — cite the paper title and year in the chapter's Further Reading.
- Vendor architecture blogs when you need a real constraint (object-store consistency, queue delivery, CDN behavior). Paraphrase the constraint; do not reproduce the article.
- Latency-order notes widely circulated from large-scale systems talks (Dean and similar lists): use orders of magnitude, not a pasted table.

Add a proper citation entry here when a chapter's Further Reading names a specific work.

SYSTEM DESIGN INTERVIEW PLAYBOOK

ABOUT THIS BOOK

System design interviews reward a method, not a catalog of logos. This playbook is for software engineers and architects who must design a service in forty-five minutes and still sound like they have operated one.

You will learn to bound the problem, size the load, draw one honest picture, and name the trade-off that holds the design together. Foundations cover requirements, capacity, scale, availability, reliability, performance, CAP, consistency, and the sentence that makes a choice defensible. Building blocks then give you caches, stores, queues, and consistent hashing. Worked problems — a URL shortener and a rate limiter — show the loop at interview speed, with original diagrams throughout.

Write it on the board. Defend it in the room.

AH Architecture Lab

Version 1.0